

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC ĐÀ LẠT**



**BÁO CÁO TIỂU LUẬN
CÔNG NGHỆ PHẦN MỀM**

Sinh viên thực hiện : Phan Trung Hiếu

MSSV: 2312616

Lớp : CTK47A

Người hướng dẫn : Gv. Lê Gia Công

Lâm Đồng, tháng 5 năm 2025

MỤC LỤC

MỤC LỤC	2
LỜI MỞ ĐẦU	4
CHƯƠNG 1. GIỚI THIỆU VỀ CÔNG NGHỆ PHẦN MỀM	5
1.1. Định nghĩa và phạm vi	5
1.2. Vòng đời phát triển phần mềm (SDLC).....	5
1.3. Vai trò trong thực tiễn	7
CHƯƠNG 2. CÁC MÔ HÌNH PHÁT TRIỂN PHẦN MỀM	8
2.1. Mô hình Thác nước (Waterfall Model).....	8
2.2. Mô hình chữ V (V-Model)	10
2.3. Mô hình tiếp cận lặp (Iterative Model)	11
2.4. Mô hình Agile/Scrum.....	12
2.5. So sánh và lựa chọn mô hình phù hợp	13
CHƯƠNG 3. QUẢN LÝ DỰ ÁN PHẦN MỀM.....	15
3.1. Khái niệm quản lý	15
3.2. Dự án là gì?	15
3.3. Quản lý dự án phần mềm	17
3.4. Quản lý rủi ro	18
3.5. Các vai trò trong dự án phần mềm	18
3.6. Vai trò của nhà quản lý dự án	18
3.7. Các yếu tố PCTS trong quản lý dự án.....	19
CHƯƠNG 4. LẤY VÀ PHÂN TÍCH YÊU CẦU.....	20
4.1. Yêu cầu là gì?	20
4.2. Các loại yêu cầu	20
4.3. Kỹ thuật thu thập yêu cầu.....	21
4.4. Tài liệu đặc tả yêu cầu phần mềm (SRS)	22
4.5. Yêu cầu người dùng và tài liệu URS.....	22
4.6. Yêu cầu chức năng và phi chức năng.....	23
4.7. Ví dụ minh họa: Hệ thống bán sách trực tuyến.....	24
CHƯƠNG 5. THIẾT KẾ KIẾN TRÚC PHẦN MỀM	26
5.1. Khái niệm kiến trúc phần mềm	26

5.2. Kiến trúc nguyên khối (Monolithic Architecture)	27
5.3. Kiến trúc N-tầng (N-tier Architecture)	28
5.4. Kiến trúc vi dịch vụ (Microservices Architecture)	29
5.5. So sánh và lựa chọn kiến trúc	30
CHƯƠNG 6. THIẾT KẾ CHI TIẾT	31
6.1. Mục tiêu của thiết kế chi tiết	31
6.2. Nội dung của thiết kế chi tiết	31
6.3. Mẫu thiết kế (Design Patterns).....	32
6.4. Kết quả đầu ra của thiết kế chi tiết.....	33
CHƯƠNG 7. CÔNG CỤ HỖ TRỢ PHÁT TRIỂN PHẦN MỀM	34
7.1. Hệ thống quản lý phiên bản	34
7.2. Git – Hệ thống quản lý phiên bản phân tán.....	35
7.3. Các khu vực làm việc của Git	36
7.4. Nguyên tắc làm việc của Git	37
7.5. Docker – Nền tảng container hóa ứng dụng.....	39
7.6. Docker Image và Docker Container.....	40
7.7. Dockerfile – Công thức xây dựng Image	41
7.8. Quy trình Build – Ship - Run	42
7.9. Docker Compose và tích hợp CI/CD	43
CHƯƠNG 8. BÀI TẬP VÀ KẾT QUẢ THỰC HÀNH	45
8.1. Bài tập và thực hành môn Công nghệ phần mềm	45
8.2. Thực hành Git.....	54
8.3. Thực hành Docker	63
KẾT LUẬN	65
TÀI LIỆU THAM KHẢO	67

LỜI MỞ ĐẦU

Trong kỷ nguyên số hóa hiện nay, phần mềm đã trở thành một thành phần không thể thiếu trong hầu hết mọi lĩnh vực của đời sống xã hội, từ giáo dục, y tế, tài chính ngân hàng đến giải trí và sản xuất công nghiệp. Sự phát triển bùng nổ của công nghệ thông tin đã tạo ra nhu cầu to lớn về các sản phẩm phần mềm chất lượng cao, đáp ứng được những yêu cầu ngày càng phức tạp của người dùng. Tuy nhiên, việc xây dựng một hệ thống phần mềm quy mô lớn, có khả năng vận hành ổn định và dễ dàng bảo trì, nâng cấp không chỉ đơn thuần là công việc lập trình.

Công nghệ phần mềm (Software Engineering) ra đời như một ngành kỹ thuật chuyên nghiệp, cung cấp các nguyên lý, phương pháp và công cụ để phát triển phần mềm một cách có hệ thống, hiệu quả và đảm bảo chất lượng. Môn học Công nghệ phần mềm trang bị cho sinh viên những kiến thức nền tảng về toàn bộ vòng đời phát triển phần mềm, từ giai đoạn phân tích yêu cầu, thiết kế, lập trình, kiểm thử cho đến triển khai và bảo trì.

Bài tiểu luận này được thực hiện với mục tiêu hệ thống hóa các kiến thức cốt lõi của môn học Công nghệ phần mềm, đồng thời tìm hiểu và thực hành các công cụ hỗ trợ phát triển phần mềm hiện đại như Git và Docker. Thông qua việc tổng hợp và phân tích các nội dung lý thuyết cũng như thực hành, tiểu luận sẽ cung cấp một cái nhìn toàn diện về quy trình phát triển phần mềm chuyên nghiệp, từ đó giúp sinh viên có được nền tảng vững chắc để áp dụng vào các dự án thực tế trong tương lai.

CHƯƠNG 1. GIỚI THIỆU VỀ CÔNG NGHỆ PHẦN MỀM

1.1. Định nghĩa và phạm vi

Công nghệ Phần mềm (CNPM) là một ngành kỹ thuật liên quan đến tất cả các khía cạnh của quá trình sản xuất phần mềm, từ giai đoạn sơ khai (lấy và phân tích yêu cầu) cho đến giai đoạn sau cùng (vận hành và bảo trì). Trong tiếng Anh, ngành này được gọi là Software Engineering (SE).

Điều quan trọng cần hiểu là CNPM không chỉ đơn thuần là lập trình. Lập trình (coding/programming) chỉ là một mắt xích trong chuỗi dài của quá trình tạo ra sản phẩm phần mềm. CNPM bao quát toàn bộ chuỗi giá trị đó, bao gồm cả con người, quy trình, công cụ và kỹ thuật.

Một định nghĩa phổ biến khác: CNPM là việc áp dụng một cách tiếp cận có hệ thống, có kỷ luật và có thể định lượng được vào sự phát triển, vận hành và bảo trì của phần mềm.

1.2. Vòng đời phát triển phần mềm (SDLC)

Quá trình sản xuất phần mềm được tổ chức thông qua các giai đoạn cụ thể, được gọi chung là Vòng đời phát triển phần mềm (Software Development Life Cycle – SDLC). Vòng đời này bao gồm 6 giai đoạn chính, mỗi giai đoạn có những công việc và mục tiêu riêng biệt:

Giai đoạn 1: Lấy và phân tích yêu cầu (Requirements Analysis)

Đây là giai đoạn khởi đầu và có ý nghĩa quyết định đối với sự thành công của toàn bộ dự án. Trong giai đoạn này, nhóm phát triển làm việc trực tiếp với khách hàng và các bên liên quan để xác định phần mềm cần phải làm gì. Các kỹ thuật thường được sử dụng bao gồm phỏng vấn, khảo sát, tổ chức hội thảo và quan sát thực tế. Kết quả của giai đoạn này là tài liệu đặc tả yêu cầu phần mềm (Software Requirements Specification – SRS), trong đó mô tả chi tiết các tính năng, giao diện và hiệu suất mà phần mềm cần đáp ứng.

Giai đoạn 2: Thiết kế (Design)

Sau khi đã có tài liệu đặc tả yêu cầu, nhóm phát triển tiến hành chuyển các yêu cầu đó thành bản vẽ kỹ thuật chi tiết. Công việc trong giai đoạn này bao gồm thiết kế kiến trúc hệ thống, thiết kế cơ sở dữ liệu, thiết kế thuật toán và giao diện người dùng. Các công cụ hỗ trợ thiết kế phổ biến bao gồm Enterprise Architect, Visual Paradigm, Lucidchart để vẽ sơ đồ UML, và Figma để thiết kế giao diện người dùng.

Giai đoạn 3: Lập trình (Implementation)

Dựa trên bản thiết kế đã được phê duyệt, các lập trình viên sử dụng các ngôn ngữ lập trình phù hợp để viết mã nguồn. Trong giai đoạn này, việc tuân thủ các tiêu chuẩn lập trình như SOLID, DRY, KISS là rất quan trọng để đảm bảo mã nguồn dễ đọc, dễ bảo trì và có khả năng mở rộng. Các công cụ phát triển tích hợp (IDE) như VS Code, Cursor, IntelliJ thường được sử dụng, cùng với hệ thống quản lý phiên bản Git để theo dõi các thay đổi.

Giai đoạn 4: Kiểm thử (Testing)

Kiểm thử là hoạt động nhằm phát hiện và sửa lỗi, đảm bảo phần mềm hoạt động đúng như yêu cầu ban đầu, ổn định và đáng tin cậy. Việc kiểm thử được thực hiện ở nhiều cấp độ khác nhau, từ kiểm thử đơn vị (unit testing), kiểm thử tích hợp (integration testing), kiểm thử hệ thống (system testing) đến kiểm thử chấp nhận (acceptance testing). Các công cụ kiểm thử phổ biến bao gồm Selenium, JMeter và Postman.

Giai đoạn 5: Triển khai (Deployment)

Sau khi phần mềm đã vượt qua các bài kiểm thử, nó sẽ được triển khai vào môi trường thực tế để khách hàng sử dụng. Giai đoạn này bao gồm các công việc như đóng gói phần mềm, thiết lập môi trường chạy, cấu hình hệ thống và hướng dẫn khách hàng sử dụng sản phẩm. Các công nghệ như Docker và các công cụ CI/CD (Jenkins, GitHub Actions) giúp quá trình triển khai diễn ra nhanh chóng và ổn định hơn.

Giai đoạn 6: Vận hành và Bảo trì (Maintenance)

Đây là giai đoạn kéo dài nhất trong vòng đời phát triển phần mềm. Sau khi phần mềm được đưa vào sử dụng, nhóm phát triển tiếp tục theo dõi hệ thống, sửa các lỗi phát sinh, cập nhật tính năng mới và nâng cấp để tương thích với những thay đổi của môi

trường công nghệ. Các hệ thống giám sát như Grafana và Prometheus được sử dụng để theo dõi hiệu suất và phát hiện sớm các vấn đề.

1.3. Vai trò trong thực tiễn

Khoảng 60 –70% các dự án phần mềm trên thế giới bị thất bại hoặc trễ tiến độ không phải vì lý do kỹ thuật, mà vì thiếu quy trình, thiếu giao tiếp và thiếu quản lý. Theo báo cáo CHAOS Report của Standish Group, chỉ khoảng 31% dự án phần mềm hoàn thành đúng hạn, đúng ngân sách và đúng yêu cầu. Điều này cho thấy kỹ năng quản lý quy trình phát triển phần mềm quan trọng không kém gì kỹ năng lập trình.

CHƯƠNG 2. CÁC MÔ HÌNH PHÁT TRIỂN PHẦN MỀM

Như đã trình bày ở Chương 1, để tạo ra một phần mềm cần trải qua 6 công đoạn cơ bản: lấy và phân tích yêu cầu, thiết kế, lập trình, kiểm thử, triển khai, và vận hành bảo trì. Tuy nhiên, trong thực tế, tùy thuộc vào quy mô và tính chất của dự án, các công đoạn này có thể được tổ chức theo nhiều cách khác nhau. Mỗi cách thức tổ chức các công đoạn của quá trình phát triển phần mềm được gọi là một Mô hình phát triển phần mềm.

Có rất nhiều mô hình phát triển phần mềm đã được đề xuất và áp dụng trong thực tế, bao gồm: mô hình thác nước, mô hình chữ V, mô hình xoắn ốc, mô hình hợp nhất, mô hình tiếp cận lặp, mô hình nguyên mẫu, mô hình Scrum, mô hình Kanban, mô hình XP. Trong phạm vi chương này, chúng ta sẽ tìm hiểu chi tiết về bốn mô hình phổ biến và được áp dụng rộng rãi nhất.

2.1. Mô hình Thác nước (Waterfall Model)

Mô hình Thác nước là một trong những mô hình phát triển phần mềm đầu tiên và truyền thống nhất. Đây là mô hình tuần tự, trong đó các giai đoạn phát triển diễn ra theo một trình tự tuyến tính, từ trên xuống dưới như một dòng thác. Mỗi giai đoạn phải được hoàn thành trước khi chuyển sang giai đoạn tiếp theo và về nguyên tắc không có sự quay lại.

Đặc điểm của mô hình Thác nước:

- Tuần tự và tuyến tính: Các giai đoạn được thực hiện lần lượt, giai đoạn sau chỉ bắt đầu khi giai đoạn trước đã hoàn thành.
- Tài liệu hóa cao: Mỗi giai đoạn đều tạo ra các tài liệu chi tiết làm đầu vào cho giai đoạn tiếp theo.
- Ít có sự tham gia của khách hàng trong quá trình phát triển: Khách hàng chủ yếu tham gia ở giai đoạn đầu (xác định yêu cầu) và giai đoạn cuối (nghiệm thu sản phẩm).

Áp dụng:

Mô hình Thác nước phù hợp với các dự án có yêu cầu cố định, đã được xác định rõ ràng ngay từ đầu và ít có khả năng thay đổi trong quá trình phát triển. Ví dụ điển hình là các dự án phát triển phần mềm nhúng, các hệ thống điều khiển trong công nghiệp, hoặc các dự án quốc phòng, nơi các yêu cầu về chức năng và an toàn được quy định chặt chẽ.

Các công đoạn cụ thể trong mô hình Thác nước

Mỗi công đoạn trong mô hình Thác nước có những công việc và công cụ hỗ trợ cụ thể:

Giai đoạn	Công việc chính	Công cụ
Lấy và phân tích yêu cầu	Làm việc với khách hàng xác định tính năng, giao diện, hiệu suất; viết tài liệu SRS	Word, Google Docs, Confluence
Thiết kế	Thiết kế kiến trúc, CSDL, thuật toán, giao diện; vẽ sơ đồ UML	Enterprise Architect, Visual Paradigm, Lucidchart, Figma
Lập trình	Viết mã theo tiêu chuẩn SOLID, DRY, KISS; viết theo unit test	VS Code, IntelliJ, Git
Kiểm thử	Kiểm thử đơn vị, tích hợp, hệ thống, chấp nhận	Selenium, JMeter, Postman
Triển khai	Đóng gói, thiết lập CI/CD, bàn giao cho khách hàng	Docker, Jenkins, Github Actions
Vận hành và bảo trì	Sửa lỗi, nâng cấp, tái cấu trúc mã nguồn	Grafana, Prometheus

Ưu điểm và nhược điểm:

Ưu điểm của mô hình Thác nước là dễ hiểu, dễ quản lý do tính tuần tự rõ ràng. Việc tài liệu hóa đầy đủ giúp cho việc bàn giao và bảo trì sau này thuận lợi hơn. Mô hình này cũng phù hợp với các dự án có quy mô nhỏ và vừa, nơi yêu cầu ổn định.

Tuy nhiên, nhược điểm lớn nhất của mô hình Thác nước là thiếu tính linh hoạt. Một khi đã chuyển sang giai đoạn sau, rất khó để quay lại sửa đổi các quyết định ở giai đoạn trước. Khách hàng chỉ nhìn thấy sản phẩm ở cuối dự án, do đó nếu có sự hiểu nhầm về yêu cầu, chi phí sửa chữa sẽ rất lớn. Ngoài ra, mô hình này không phù hợp với các dự án có yêu cầu thay đổi thường xuyên hoặc chưa được xác định rõ ràng.

2.2. Mô hình chữ V (V-Model)

Mô hình chữ V là một biến thể của mô hình Thác nước, được phát triển nhằm nhấn mạnh tầm quan trọng của hoạt động kiểm thử. Trong mô hình này, các giai đoạn phát triển và các giai đoạn kiểm thử được thực hiện song song và có sự tương ứng với nhau. Mỗi giai đoạn phát triển sẽ có một giai đoạn kiểm thử tương ứng.

Đặc điểm của mô hình chữ V:

- Nhấn mạnh kiểm thử: Kiểm thử được lên kế hoạch song song với phát triển, không phải đợi đến cuối dự án.
- Mối quan hệ tương ứng: Giai đoạn phân tích yêu cầu tương ứng với kiểm thử chấp nhận, thiết kế kiến trúc tương ứng với kiểm thử hệ thống, thiết kế chi tiết tương ứng với kiểm thử tích hợp và lập trình tương ứng với kiểm thử đơn vị.
- Xác minh và thẩm định: Mô hình này tích hợp cả hai hoạt động xác minh (verification) và thẩm định (validation).

Áp dụng:

Mô hình chữ V phù hợp cho các dự án yêu cầu độ tin cậy cao và ít có sự thay đổi về yêu cầu, ví dụ như các hệ thống y tế, các hệ thống kiểm soát công nghiệp, hoặc phần mềm nhúng trong các thiết bị an toàn.

Ưu điểm và nhược điểm:

Ưu điểm của mô hình chữ V là giúp phát hiện lỗi sớm hơn trong quy trình phát triển, giảm thiểu rủi ro và chi phí sửa lỗi. Việc có kế hoạch kiểm thử rõ ràng ngay từ đầu giúp đảm bảo chất lượng sản phẩm cao hơn.

Nhược điểm của mô hình cũng tương tự như mô hình Thác nước: thiếu tính linh hoạt và khó thích ứng với những thay đổi về yêu cầu. Ngoài ra, việc tạo ra các tài liệu kiểm thử chi tiết ngay từ đầu đòi hỏi nhiều về thời gian và công sức.

2.3. Mô hình tiếp cận lặp (Iterative Model)

Mô hình tiếp cận lặp khắc phục nhược điểm thiếu linh hoạt của các mô hình tuần tự bằng cách chia dự án thành nhiều lần lặp (iteration) nhỏ. Mỗi lần lặp sẽ trải qua đầy đủ các giai đoạn của vòng đời phát triển phần mềm (phân tích, thiết kế, lập trình, kiểm thử) và tạo ra một phiên bản sản phẩm có thể hoạt động được. Qua mỗi lần lặp, sản phẩm được bổ sung thêm tính năng và hoàn thiện dần dựa trên phản hồi từ người dùng.

Đặc điểm của mô hình tiếp cận lặp:

- Phát triển theo chu kỳ: Dự án được chia thành nhiều lần lặp, mỗi lần lặp kéo dài từ vài tuần đến vài tháng.
- Phản hồi liên tục: Sau mỗi lần lặp, khách hàng có thể xem xét và đưa ra phản hồi về sản phẩm, từ đó điều chỉnh hướng phát triển cho các lần lặp tiếp theo
- Giảm rủi ro: Việc phát hành sản phẩm sớm (dù chưa đầy đủ tính năng) giúp phát hiện sớm các vấn đề về yêu cầu và thiết kế.

Áp dụng:

Mô hình tiếp cận lặp phù hợp với các dự án mà yêu cầu không rõ ràng ngay từ đầu, hoặc khi cần phát hành sản phẩm sớm để lấy ý kiến phản hồi từ người dùng. Đây là mô hình rất phổ biến trong phát triển các ứng dụng web và di động, nơi thị trường và nhu cầu người dùng thay đổi nhanh chóng.

Ưu điểm và nhược điểm:

Ưu điểm của mô hình tiếp cận lặp là tính linh hoạt cao, cho phép điều chỉnh yêu cầu và thiết kế dựa trên phản hồi thực tế. Rủi ro được giảm thiểu do các vấn đề được

phát hiện sớm. Khách hàng có thể nhìn thấy tiến độ và sản phẩm một cách thường xuyên, tạo sự tin tưởng và hài lòng.

Nhược điểm của mô hình này là khó ước tính chính xác tổng chi phí và thời gian do yêu cầu có thể thay đổi trong quá trình phát triển. Việc quản lý nhiều lần lặp cũng đòi hỏi kỹ năng quản lý dự án tốt hơn so với mô hình tuần tự.

2.4. Mô hình Agile/Scrum

Agile là một triết lý phát triển phần mềm đề cao sự linh hoạt, hợp tác và phản hồi nhanh với thay đổi. Scrum là một trong những framework phổ biến nhất để triển khai Agile. Scrum dựa trên việc chia dự án thành các chu kỳ phát triển ngắn (thường 1-4 tuần) gọi là Sprint. Trong mỗi Sprint, đội nhóm sẽ hoàn thành một tập hợp các tính năng được ưu tiên, tạo ra một phiên bản sản phẩm có thể hoạt động và sẵn sàng để trình diễn.

Đặc điểm của mô hình Scrum:

- **Sprint:** Là chu kỳ phát triển ngắn, có độ dài cố định (thường 1-4 tuần). Mỗi Sprint đều có mục tiêu rõ ràng và kết thúc bằng một phiên bản sản phẩm có thể hoạt động.
- **Product Backlog:** Là danh sách tất cả các tính năng, yêu cầu, cải tiến và sửa lỗi cần thực hiện cho sản phẩm, được sắp xếp theo thứ tự ưu tiên.
- **Sprint Backlog:** Là tập hợp con của Product Backlog, bao gồm các hạng mục được chọn để thực hiện trong Sprint hiện tại.
- **Các vai trò chính:** Product Owner (chịu trách nhiệm về Product Backlog và tối đa hóa giá trị sản phẩm), Scrum Master (đảm bảo quy trình Scrum được tuân thủ và loại bỏ các trở ngại) và Development Team (nhóm phát triển tự tổ chức).
- **Các sự kiện:** Sprint Planning (lập kế hoạch Sprint), Daily Scrum (họp hàng ngày 15 phút), Sprint Review (đánh giá kết quả Sprint) và Sprint Retrospective (cải tiến quy trình).

Áp dụng:

Scrum phù hợp với các dự án có yêu cầu thường xuyên thay đổi và cần sự linh hoạt cao. Sự hợp tác chặt chẽ giữa các thành viên trong nhóm và với khách hàng là yếu tố then chốt. Scrum đặc biệt hiệu quả trong phát triển các sản phẩm phần mềm sáng tạo, nơi yêu cầu có thể thay đổi nhanh chóng dựa trên phản hồi thị trường.

Ưu điểm và nhược điểm:

Ưu điểm của Scrum là khả năng thích ứng cao với thay đổi, tạo ra giá trị sớm và liên tục cho khách hàng. Nhóm phát triển có quyền tự chủ cao, tạo động lực và nâng cao năng suất. Các sự kiện Scrum giúp đảm bảo tính minh bạch và cải tiến liên tục.

Nhược điểm của Scrum là đòi hỏi sự cam kết và kỷ luật cao từ tất cả thành viên trong nhóm. Nếu không được áp dụng đúng cách, Scrum có thể dẫn đến tình trạng “Scrum - but” (chỉ áp dụng một lần) làm giảm hiệu quả. Ngoài ra, Scrum có thể không phù hợp với các dự án có yêu cầu cố định và không thay đổi.

2.5. So sánh và lựa chọn mô hình phù hợp

Mỗi mô hình phát triển phần mềm đều có những ưu điểm và hạn chế riêng, phù hợp với những loại dự án khác nhau. Việc lựa chọn mô hình phù hợp cần dựa trên nhiều yếu tố, bao gồm:

- Tính ổn định của yêu cầu: Nếu yêu cầu rõ ràng và ổn định, mô hình Thác nước hoặc chữ V có thể phù hợp. Nếu yêu cầu chưa rõ ràng hoặc có khả năng thay đổi, mô hình lặp hoặc Scrum sẽ tốt hơn.
- Quy mô và độ phức tạp của dự án: Dự án lớn và phức tạp thường phù hợp với các mô hình có cấu trúc như Thác nước, trong khi dự án nhỏ và vừa có thể áp dụng Scrum hiệu quả.
- Mức độ tham gia của khách hàng: Nếu khách hàng có thể tham gia thường xuyên, Scrum là lựa chọn tốt. Nếu khách hàng chỉ tham gia ở đầu và cuối dự án, Thác nước phù hợp hơn.
- Yêu cầu về chất lượng và độ tin cậy: Các hệ thống yêu cầu độ tin cậy cao (y tế, hàng không) thường áp dụng mô hình chữ V với quy trình kiểm thử nghiêm ngặt.

- Văn hóa và kinh nghiệm của nhóm: Nhóm có kinh nghiệm với Scrum sẽ làm việc hiệu quả hơn với mô hình này.

Trong thực tế, nhiều tổ chức kết hợp các yếu tố từ nhiều mô hình khác nhau để tạo ra quy trình phát triển phù hợp nhất với đặc thù của mình.

CHƯƠNG 3. QUẢN LÝ DỰ ÁN PHẦN MỀM

Ở chương trước, chúng ta đã tìm hiểu về các mô hình phát triển phần mềm – những phương thức tổ chức các công đoạn để tạo ra một sản phẩm phần mềm. Tuy nhiên, trong thực tế, quá trình xây dựng phần mềm không chỉ gói gọn trong nội bộ nhóm phát triển. Một dự án luôn có sự tham gia và tác động từ nhiều bên liên quan như khách hàng, nhà đầu tư và những người quản lý. Nhiệm vụ của chúng ta không chỉ dừng lại ở việc hoàn thiện các tính năng kỹ thuật, mà còn phải đảm bảo tính hiệu quả về mặt kinh tế và hoàn thành đúng thời hạn quy định.

Để có thể cân bằng được lợi ích của các bên liên quan, đồng thời đảm bảo sản phẩm ra đời đầy đủ chức năng, đúng tiến độ và nằm trong phạm vi ngân sách cho phép, chúng ta cần đến kiến thức chuyên sâu về Quản lý dự án phần mềm.

3.1. Khái niệm quản lý

Quản lý (nói chung) là sự tác động của chủ thể quản lý lên đối tượng quản lý, nhằm đạt được mục tiêu nhất định, trong điều kiện biến động của môi trường. Các yếu tố cấu thành hoạt động quản lý bao gồm: có chủ thể quản lý (người quản lý), có đối tượng quản lý (người bị quản lý), có mục tiêu cần đạt được, và có môi trường quản lý.

Quản lý có thể được nhìn nhận từ nhiều góc độ khác nhau. Một số quan điểm cho rằng quản lý là nghệ thuật đạt được mục đích thông qua nỗ lực của những người khác. Quan điểm khác xem quản lý là công tác phối hợp có hiệu quả các hoạt động của những cộng sự khác nhau trong cùng một tổ chức. Quản lý cũng được định nghĩa là quá trình lập kế hoạch, tổ chức, chỉ huy, lãnh đạo và kiểm tra các nguồn lực cơ quan, nhằm đạt được mục đích với hiệu quả cao, trong điều kiện môi trường luôn biến động.

Quản lý được xem vừa là một nghệ thuật, vừa là một khoa học, và là một nghề. Tính nghệ thuật thể hiện ở sự đa dạng, phong phú của các tình huống quản lý và cách ứng xử linh hoạt của nhà quản lý. Tính khoa học thể hiện ở việc vận dụng các quy luật kinh tế, công nghệ, xã hội và các thành tựu khoa học trong quản lý. Là một nghề, quản lý đòi hỏi phải được đào tạo và có năng khiếu nghề nghiệp.

3.2. Dự án là gì?

Dự án là một nỗ lực có thời hạn nhằm tạo ra một sản phẩm, một dịch vụ, hoặc một kết quả duy nhất. Một dự án chỉ được triển khai một lần; nếu việc đó được lặp lại thì nó không được coi là dự án nữa.

Bốn yếu tố quan trọng của một dự án, thường được gọi là PCTS (Performance, Cost, Time, Scope – Chất lượng thực hiện, Chi phí, Thời hạn, Quy mô), bao gồm:

- **Chất lượng thực hiện (Performance):** Tập thể thực hiện dự án và chất lượng công việc của họ.
- **Chi phí (Cost):** Kinh phí dự kiến để thực hiện dự án.
- **Thời hạn (Time):** Thời gian dự kiến để thực hiện dự án, với thời điểm bắt đầu và kết thúc cụ thể.
- **Quy mô (Scope):** Kết quả dự kiến cần đạt được.

Bốn yếu tố này ràng buộc lẫn nhau: thay đổi một yếu tố sẽ ảnh hưởng đến các yếu tố còn lại. Ví dụ: muốn rút ngắn thời gian mà không thay đổi phạm vi thì phải tăng chi phí (thêm nhân lực) hoặc chấp nhận giảm chất lượng.

Ví dụ, một dự án "xây dựng một ngôi nhà" có các yếu tố: tập thể thực hiện gồm kiến trúc sư, công nhân xây dựng, giám sát; thời gian dự kiến 6 tháng; kinh phí dự kiến 1 tỷ đồng; kết quả cần đạt được là một ngôi nhà 3 tầng, đầy đủ tiện nghi cho một gia đình sinh sống.

Các thuộc tính của dự án bao gồm:

- Dự án có mục đích rõ ràng.
- Dự án mang tính tạm thời: có ngày bắt đầu và ngày kết thúc.
- Dự án sử dụng các loại tài nguyên khác nhau: con người, phần cứng, phần mềm, công cụ, thiết bị.
- Dự án phải có khách hàng và/hoặc nhà tài trợ.
- Dự án thường mang tính không chắc chắn, nghĩa là để đạt được các mục tiêu trong một khoảng thời gian, với một số tiền và nhân lực cho trước là một công việc nhiều thách thức.

3.3. Quản lý dự án phần mềm

Quản lý dự án là hoạt động áp dụng các kiến thức, kỹ năng, công cụ và kỹ thuật để lên kế hoạch hành động, nhằm đạt được các yêu cầu của dự án. Công tác quản lý dự án được thực hiện thông qua việc áp dụng và phối hợp 5 nhóm quy trình gồm: Khởi xướng, Lập kế hoạch, Triển khai thực hiện, Giám sát và kiểm soát, Kết thúc.

Vai trò của quản lý dự án

Quản lý dự án hiệu quả mang lại nhiều lợi ích quan trọng:

- Kiểm soát tốt hơn các tài nguyên: nhân lực, tài chính, vật liệu.
- Cải tiến quan hệ với khách hàng.
- Rút ngắn thời gian phát triển.
- Giảm chi phí, tăng lợi nhuận.
- Tăng chất lượng và độ tin cậy của sản phẩm.
- Cải tiến năng suất lao động.
- Phối hợp nội bộ tốt hơn.
- Tinh thần làm việc cao hơn.
- Các công cụ và kỹ thuật trong quản lý dự án

Để thực hiện quản lý dự án hiệu quả, nhà quản lý cần sử dụng nhiều công cụ và kỹ thuật khác nhau:

- Project charter (tôn chỉ dự án), scope statement (bản tuyên bố phạm vi), WBS (Work Breakdown Structure – Cấu trúc phân rã công việc).
- Biểu đồ Gantt, biểu đồ mạng, phân tích đường tới hạn.
- Ước lượng chi phí và quản trị giá trị đạt được.

Các công việc trong quản lý dự án

Các công việc chính trong quản lý dự án bao gồm:

- Nhận diện các yêu cầu của dự án.

- Cân đối được các nhu cầu, mối quan tâm, mong đợi khác nhau từ các thành phần liên quan (stakeholder) khi lên kế hoạch và thực thi dự án.

- Cân đối giữa các ràng buộc có tính cạnh tranh như: phạm vi (scope), chất lượng (quality), lịch biểu (schedule), ngân sách (budget), tài nguyên (resource), rủi ro (risk).

3.4. Quản lý rủi ro

Rủi ro phổ biến trong dự án phần mềm:

- Yêu cầu thay đổi liên tục (requirements creep)
- Thiếu nhân lực có kỹ năng phù hợp
- Ước tính thời gian không chính xác (thường bị đánh giá thấp hơn thực tế)
- Nợ kỹ thuật (technical debt) tích lũy theo thời gian
- Sự phụ thuộc vào công nghệ hoặc nhà cung cấp bên thứ ba
- Giao tiếp kém giữa các bên liên quan

3.5. Các vai trò trong dự án phần mềm

- Product Owner / Business Analyst: Đại diện khách hàng, xác định và ưu tiên các tính năng, viết User Stories

- Project Manager / Scrum Master: Lập kế hoạch, theo dõi tiến độ, tháo gỡ rào cản cho nhóm

- Solution Architect: Thiết kế kiến trúc tổng thể, đưa ra các quyết định kỹ thuật quan trọng

- Software Developer: Viết mã nguồn, thực hiện code review, viết unit test

- QA Engineer / Tester: Thiết kế và thực thi kịch bản kiểm thử, báo cáo lỗi

- DevOps Engineer: Xây dựng và vận hành pipeline CI/CD, quản lý hạ tầng

- UI/UX Designer: Thiết kế giao diện người dùng, đảm bảo trải nghiệm sử dụng tốt

3.6. Vai trò của nhà quản lý dự án

Nhà quản lý dự án đóng vai trò then chốt trong sự thành công của dự án. Một nguyên tắc quan trọng là nhà quản lý dự án không nên tham gia vào phần việc cụ thể của dự án, nghĩa là không nên vừa quản lý dự án vừa thực hiện dự án, vì khi đó họ sẽ bị "giằng xé" giữa trách nhiệm quản lý và trách nhiệm phải hoàn thành phần việc của mình.

Nhà quản lý dự án đóng vai trò của một tác nhân (enabler). Công việc của họ là khuyến khích đội dự án hoàn thành công việc, giúp đội giải quyết các khó khăn, đáp ứng các nguồn lực khan hiếm mà đội cần và che chắn cho họ khỏi sự cản trở của các thế lực bên ngoài. Họ không phải là "ông vua" của dự án, mà phải là một nhà lãnh đạo thực thụ. Nghĩa là quản lý dự án không đơn giản chỉ là việc lập kế hoạch, xây dựng tiến độ và giám sát công việc.

3.7. Các yếu tố PCTS trong quản lý dự án

Một trong những nguyên nhân phổ biến dẫn đến thất bại của các dự án phần mềm là do nhà tài trợ yêu cầu nhà quản lý dự án phải hoàn thành công việc trong một thời gian nhất định, với ngân sách được cấp và ở quy mô xác định, trong khi vẫn phải đạt được chất lượng thực hiện cụ thể. Nói cách khác, nhà tài trợ quyết định cả bốn yêu cầu (PCTS) ràng buộc của dự án. Cách tiếp cận này không thể đem lại kết quả.

Thực tế cho thấy khoảng 83% dự án phần mềm gặp vấn đề liên quan đến mục tiêu PCTS. Điều này cho thấy tầm quan trọng của việc quản lý dự án chuyên nghiệp và sự cần thiết phải có sự cân đối hợp lý giữa các yếu tố ràng buộc.

Quản lý dự án phần mềm là một lĩnh vực phức tạp, đòi hỏi sự kết hợp giữa kiến thức chuyên môn về công nghệ và kỹ năng quản lý. Một nhà quản lý dự án giỏi cần hiểu rõ về quy trình phát triển phần mềm, có khả năng giao tiếp hiệu quả với các bên liên quan, và biết cách tạo động lực cho đội ngũ phát triển. Trong bối cảnh công nghệ thay đổi nhanh chóng, vai trò của quản lý dự án ngày càng trở nên quan trọng, đảm bảo rằng các dự án phần mềm được thực hiện đúng hướng, đúng tiến độ và mang lại giá trị thực sự cho khách hàng.

CHƯƠNG 4. LẤY VÀ PHÂN TÍCH YÊU CẦU

Sau khi đã lựa chọn được mô hình phát triển phần mềm phù hợp và có kiến thức về quản lý dự án, bước tiếp theo trong quy trình phát triển phần mềm là giai đoạn Lấy và phân tích yêu cầu. Đây là giai đoạn đầu tiên và cũng là một trong những giai đoạn quan trọng nhất của vòng đời phát triển phần mềm. Chất lượng của giai đoạn này ảnh hưởng trực tiếp đến sự thành công của toàn bộ dự án.

4.1. Yêu cầu là gì?

Trong phát triển phần mềm, yêu cầu (requirements) là những mô tả về các điều kiện, khả năng, và tính năng mà một hệ thống phần mềm phải có để đáp ứng nhu cầu của người dùng hoặc các bên liên quan. Nói một cách đơn giản, yêu cầu là cầu nối giữa mong muốn của khách hàng và sản phẩm cuối cùng.

Yêu cầu là nền tảng cho toàn bộ quá trình phát triển phần mềm. Nếu không có các yêu cầu rõ ràng và chính xác, dự án sẽ gặp rủi ro lớn vì nhóm phát triển có thể xây dựng một sản phẩm không đáp ứng được mục đích ban đầu, dẫn đến lãng phí thời gian và nguồn lực. Nhiều nghiên cứu đã chỉ ra rằng phần lớn các dự án phần mềm thất bại đều có nguyên nhân bắt nguồn từ việc quản lý yêu cầu kém.

4.2. Các loại yêu cầu

Trong phát triển phần mềm, chúng ta quan tâm tới ba loại yêu cầu chính: yêu cầu người dùng (user requirements), yêu cầu nghiệp vụ (domain requirements), và phi yêu cầu (non-requirements).

Yêu cầu người dùng (User Requirements)

Yêu cầu người dùng là những nhu cầu, mong đợi, và kỳ vọng của người dùng cuối về một sản phẩm, hệ thống, hoặc dịch vụ. Nó mô tả những gì người dùng mong đợi hệ thống thực hiện để giải quyết vấn đề của họ. Yêu cầu người dùng chia thành hai loại: chức năng (tính năng của hệ thống) và phi chức năng (tốc độ, hiệu suất, tin cậy).

- **Yêu cầu chức năng (Functional Requirements):** mô tả các tính năng hệ thống cần có. Ví dụ: "Hệ thống phải cho phép người dùng đăng nhập bằng email và mật khẩu"

- Yêu cầu phi chức năng (Non-Functional Requirements – NFR): mô tả các đặc tính về tốc độ, bảo mật, độ tin cậy. Ví dụ: "Hệ thống phải phản hồi trong vòng 2 giây với 10.000 người dùng đồng thời"

Yêu cầu người dùng được viết bằng ngôn ngữ tự nhiên, dễ hiểu, không mang tính kỹ thuật. Chúng thường được thu thập thông qua phỏng vấn, khảo sát, và các buổi họp với khách hàng hoặc người dùng cuối. Yêu cầu này thường được ghi lại trong một tài liệu gọi là Tài liệu đặc tả yêu cầu người dùng (User Requirement Specification – URS) để làm cơ sở cho việc thiết kế và phát triển sản phẩm.

Yêu cầu nghiệp vụ (Domain Requirements)

Yêu cầu nghiệp vụ là những yêu cầu xuất phát từ lĩnh vực hoạt động cụ thể của hệ thống. Chúng phản ánh các quy tắc, quy định, và đặc thù của ngành nghề mà hệ thống sẽ hoạt động. Ví dụ, trong lĩnh vực ngân hàng, phần mềm phải tuân thủ các quy định của Ngân hàng Nhà nước về bảo mật thông tin khách hàng. Đây là những ràng buộc mà nhóm phát triển phải tuân theo dù khách hàng có đề cập hay không.

Phi yêu cầu (Non-requirements)

Phi yêu cầu là những điều mà hệ thống không được phép làm hoặc những giới hạn rõ ràng về phạm vi của hệ thống. Việc xác định rõ những gì nằm ngoài phạm vi dự án giúp tránh hiểu nhầm và "phình to phạm vi" (scope creep) sau này.

4.3. Kỹ thuật thu thập yêu cầu

- Phỏng vấn (Interview): Trực tiếp hỏi các bên liên quan để hiểu nhu cầu sâu nhất. Chuẩn bị câu hỏi cẩn thận, ghi chép đầy đủ.
- Khảo sát (Survey/Questionnaire): Thu thập ý kiến từ số lượng lớn người dùng trong thời gian ngắn.
- Quan sát (Observation): Theo dõi người dùng thực hiện công việc để phát hiện nhu cầu ngầm mà họ không tự diễn đạt được.
- Workshop / Focus Group: Tổ chức buổi họp nhóm để thảo luận và đồng thuận về yêu cầu.

- Phân tích tài liệu (Document Analysis): Nghiên cứu hệ thống hiện tại, quy trình nghiệp vụ, hợp đồng, báo cáo.
- Nguyên mẫu (Prototyping): Xây dựng bản mẫu thô để khách hàng xem và phản hồi trước khi phát triển thật.
- Viết User Stories: Mô tả yêu cầu theo cấu trúc: "Là [ai], tôi muốn [làm gì], để [đạt mục đích gì]"

4.4. Tài liệu đặc tả yêu cầu phần mềm (SRS)

Tài liệu đặc tả yêu cầu phần mềm (Software Requirements Specification – SRS) là tài liệu kỹ thuật chính thức, mô tả đầy đủ các yêu cầu của hệ thống. Đây là "hợp đồng kỹ thuật" giữa nhóm phát triển và khách hàng, là cơ sở để thiết kế, lập trình và kiểm thử.

Một SRS tốt cần đảm bảo:

- Hoàn chỉnh (Complete): Đủ tất cả yêu cầu, không bỏ sót
- Nhất quán (Consistent): Không mâu thuẫn nội bộ
- Khả kiểm tra (Testable/Verifiable): Mỗi yêu cầu có thể kiểm thử được
- Có thể theo dõi (Traceable): Mỗi yêu cầu có mã ID riêng để truy vết
- Khả thi (Feasible): Có thể thực hiện được trong điều kiện ràng buộc hiện có

4.5. Yêu cầu người dùng và tài liệu URS

Tài liệu đặc tả yêu cầu người dùng (URS) có mục đích và đặc điểm quan trọng sau:

Mục đích của yêu cầu người dùng

- Định hướng phát triển: Giúp đội ngũ phát triển và kỹ sư hiểu rõ nhu cầu của người dùng để xây dựng sản phẩm đúng mong muốn.
- Đảm bảo sự hài lòng: Sản phẩm cuối cùng sẽ đáp ứng các kỳ vọng của người dùng, mang lại trải nghiệm tốt và sự hài lòng.

- Quản lý dự án: Là cơ sở để lập kế hoạch, ước tính chi phí, kiểm soát dự án và đánh giá thành công.

Đặc điểm của yêu cầu người dùng

- Từ góc độ người dùng: Tài liệu URS được viết theo góc nhìn của người dùng cuối, không quá kỹ thuật hay phức tạp, và để người dùng có kiến thức chung về hệ thống có thể hiểu được.

- Chức năng và phi chức năng: Bao gồm các tính năng mà hệ thống cần có (chức năng) và các yếu tố khác như hiệu suất, tốc độ, độ tin cậy (phi chức năng).

- Làm cơ sở cho tài liệu kỹ thuật: Là nền tảng cho các tài liệu đặc tả kỹ thuật do đội ngũ phát triển tạo ra, quy định cách thức đáp ứng yêu cầu của người dùng.

4.6. Yêu cầu chức năng và phi chức năng

Yêu cầu chức năng (Functional Requirements)

Yêu cầu chức năng là các tính năng cụ thể mà một hệ thống phải thực hiện để đáp ứng nhu cầu của người dùng. Chúng mô tả hệ thống "làm gì".

Ví dụ về yêu cầu chức năng:

- Hệ thống phải cho phép người dùng đăng ký tài khoản.
- Hệ thống phải cho phép người dùng tìm kiếm sản phẩm theo từ khóa.
- Hệ thống phải gửi email xác nhận khi người dùng đặt hàng thành công.

Yêu cầu phi chức năng (Non-Functional Requirements)

Yêu cầu phi chức năng mô tả các thuộc tính chất lượng của hệ thống, như hiệu suất, bảo mật, khả năng mở rộng, độ tin cậy. Chúng mô tả hệ thống "như thế nào" khi thực hiện các chức năng.

Ví dụ về yêu cầu phi chức năng:

- Hệ thống phải phản hồi trong vòng 2 giây đối với 95% các yêu cầu.
- Hệ thống phải có khả năng phục vụ đồng thời 10.000 người dùng.
- Dữ liệu người dùng phải được mã hóa khi lưu trữ và truyền tải.

4.7. Ví dụ minh họa: Hệ thống bán sách trực tuyến

Để minh họa cho việc xác định yêu cầu chức năng, chúng ta xem xét ví dụ về một hệ thống bán sách trực tuyến. Các yêu cầu được phân thành ba nhóm chính:

1. Yêu cầu liên quan đến người dùng

Các yêu cầu này tập trung vào việc quản lý thông tin và tương tác của người dùng với hệ thống.

Đăng ký và đăng nhập:

FR-1.1: Hệ thống phải cho phép người dùng mới đăng ký tài khoản bằng cách cung cấp email, tên đầy đủ và mật khẩu.

FR-1.2: Hệ thống phải xác thực thông tin đăng nhập của người dùng đã có tài khoản.

FR-1.3: Hệ thống phải cung cấp tính năng "Quên mật khẩu", cho phép người dùng đặt lại mật khẩu qua email.

Quản lý tài khoản cá nhân:

FR-1.4: Người dùng phải có khả năng cập nhật thông tin cá nhân của họ, bao gồm tên, địa chỉ, số điện thoại.

FR-1.5: Người dùng phải có thể xem lịch sử mua hàng của họ, bao gồm các đơn hàng đã đặt, trạng thái đơn hàng và chi tiết các sản phẩm đã mua.

2. Yêu cầu liên quan đến sản phẩm và tìm kiếm

Những yêu cầu này mô tả cách người dùng tương tác với danh mục sản phẩm.

Tìm kiếm và lọc sản phẩm:

FR-2.1: Hệ thống phải cho phép người dùng tìm kiếm sách theo tiêu đề, tác giả, hoặc nhà xuất bản.

FR-2.2: Người dùng phải có khả năng lọc kết quả tìm kiếm theo nhiều tiêu chí như thể loại (tiểu thuyết, kinh tế, khoa học), giá cả, hoặc đánh giá của người dùng.

Hiển thị thông tin sản phẩm:

FR-2.3: Hệ thống phải hiển thị một trang chi tiết cho mỗi cuốn sách, bao gồm hình ảnh bìa, mô tả, giá, tác giả, và các thông số kỹ thuật khác (số trang, năm xuất bản).

FR-2.4: Trang chi tiết sản phẩm phải hiển thị các đánh giá và nhận xét từ những người dùng khác.

3. Yêu cầu liên quan đến giỏ hàng và thanh toán

Các yêu cầu này mô tả quy trình mua hàng, từ khi thêm sản phẩm vào giỏ đến khi hoàn tất thanh toán.

Quản lý giỏ hàng:

FR-3.1: Người dùng phải có khả năng thêm một hoặc nhiều cuốn sách vào giỏ hàng.

FR-3.2: Người dùng có thể chỉnh sửa số lượng sách trong giỏ hàng hoặc xóa sách ra khỏi giỏ.

FR-3.3: Giỏ hàng phải tự động cập nhật tổng giá trị đơn hàng khi có thay đổi về số lượng hoặc sản phẩm.

Thanh toán và Đặt hàng:

FR-3.4: Hệ thống phải cho phép người dùng lựa chọn phương thức thanh toán.

FR-3.5: Hệ thống phải gửi một email xác nhận đơn hàng tới người dùng sau khi thanh toán thành công.

Thông qua ví dụ trên, chúng ta thấy rằng việc xác định yêu cầu một cách chi tiết và có cấu trúc sẽ tạo nền tảng vững chắc cho các giai đoạn thiết kế và phát triển tiếp theo. Các yêu cầu được đánh số và phân loại rõ ràng giúp dễ dàng theo dõi, quản lý và đảm bảo không bỏ sót tính năng nào trong quá trình phát triển.

CHƯƠNG 5. THIẾT KẾ KIẾN TRÚC PHẦN MỀM

Sau khi đã hoàn thành giai đoạn lấy và phân tích yêu cầu, chúng ta đã có một tài liệu mô tả chi tiết những gì hệ thống cần phải làm. Bước tiếp theo trong quy trình phát triển phần mềm là thiết kế – chuyển những yêu cầu đó thành bản vẽ kỹ thuật để có thể triển khai lập trình. Thiết kế phần mềm thường được chia thành hai cấp độ: thiết kế kiến trúc (cấp cao) và thiết kế chi tiết (cấp thấp). Trong chương này, chúng ta sẽ tập trung vào thiết kế kiến trúc phần mềm.

5.1. Khái niệm kiến trúc phần mềm

Kiến trúc phần mềm là bản thiết kế tổng thể của một hệ thống, giúp chúng ta hiểu rõ các bộ phận cấu thành, cách chúng kết nối và các quy tắc chi phối toàn bộ cấu trúc. Nó giống như bản vẽ kỹ thuật của một tòa nhà lớn, mô tả tòa nhà có bao nhiêu tầng, vật liệu chính là gì, hệ thống điện nước chạy như thế nào, mô tả các bộ phận (móng, cột, tường, mái), cách chúng được liên kết (hệ thống ống nước, dây điện) và mục tiêu chất lượng (chịu được động đất, tiết kiệm năng lượng).

Kiến trúc phần mềm trả lời câu hỏi: "Chúng ta nên xây dựng hệ thống này như thế nào?" thay vì "Hệ thống này làm được gì?" (câu hỏi của tài liệu URS).

Kiến trúc phần mềm mô tả ba yếu tố chính của một hệ thống:

- **Cấu trúc (Structure):** Hệ thống được chia thành các thành phần (components) hoặc mô-đun nhỏ hơn (ví dụ: mô-đun quản lý người dùng, mô-đun xử lý thanh toán).
- **Tương tác (Interaction):** Các thành phần kết nối và trao đổi dữ liệu với nhau bằng cách nào (ví dụ: dùng API).
- **Yếu tố chất lượng (Quality Attributes):** Đây là các yêu cầu phi chức năng mà kiến trúc phải đáp ứng, như tốc độ, bảo mật, khả năng mở rộng, độ tin cậy.

Vai trò quan trọng của Kiến trúc phần mềm

- **Tính ổn định:** Đảm bảo hệ thống vững chắc, không bị sụp đổ khi lượng người dùng tăng lên.

- Tính dễ hiểu: Giúp các lập trình viên mới hoặc các nhóm khác nhau dễ dàng nắm bắt được cách hệ thống hoạt động, từ đó đẩy nhanh quá trình phát triển, nâng cấp và sửa lỗi.

- Tính mở rộng: Cho phép thêm tính năng hoặc tăng khả năng chịu tải một cách dễ dàng mà không cần phải phát triển lại toàn bộ hệ thống.

5.2. Kiến trúc nguyên khối (Monolithic Architecture)

Kiến trúc nguyên khối là mô hình truyền thống, trong đó toàn bộ ứng dụng được xây dựng và triển khai dưới dạng một khối duy nhất. Tất cả các thành phần của ứng dụng như giao diện người dùng, logic nghiệp vụ và truy cập dữ liệu đều được tích hợp chặt chẽ trong cùng một ứng dụng.

Ưu điểm

- Quá trình phát triển, triển khai đơn giản: Do tất cả nằm trong một khối duy nhất, việc phát triển, thử nghiệm và triển khai sẽ đơn giản hơn.
- Hiệu suất cao: Do không có giao tiếp liên bộ phận, ứng dụng thường có hiệu suất cao hơn so với các kiến trúc phân tán.

Nhược điểm

- Khó bảo trì và mở rộng: Khi ứng dụng phát triển lớn hơn, việc bảo trì và thêm tính năng mới trở nên phức tạp.
- Khả năng chịu lỗi kém: Một lỗi nhỏ có thể làm gián đoạn toàn bộ hệ thống.

Khi nào nên sử dụng mô hình nguyên khối

- Dự án nhỏ, cần triển khai nhanh.
- Đội ngũ phát triển ít thành viên.
- Tài nguyên, máy móc ít.
- Việc giao tiếp giữa các thành phần của ứng dụng cần hiệu suất cao.

Ví dụ: Hầu hết các ứng dụng desktop truyền thống đều được thiết kế theo kiểu kiến trúc nguyên khối. Các hệ quản trị nội dung (CMS) cũ như WordPress, hoặc các ứng dụng thương mại điện tử nhỏ cũng thường sử dụng kiến trúc này.

5.3. Kiến trúc N-tầng (N-tier Architecture)

Mô hình N-tầng phân chia ứng dụng thành các tầng độc lập, chạy trên các máy chủ hoặc dịch vụ khác nhau. Sự phân tách này đạt được thông qua việc giới hạn sự giao tiếp: mỗi tầng chỉ được phép giao tiếp với tầng liền kề nó.

Mô hình N-tầng phổ biến nhất là Kiến trúc 3-tầng (3-tier architecture), phân chia ứng dụng thành ba tầng riêng biệt: Trình bày (Presentation), Logic nghiệp vụ (Business Logic), và Dữ liệu (Data).

Tên tầng	Vai trò chính	Nhiệm vụ	Ví dụ công nghệ
Trình bày (Presentation)	Giao diện người dùng	Xử lý việc hiển thị thông tin, nhận yêu cầu từ người dùng và chuyển tiếp xuống tầng Logic nghiệp vụ	React, Angular, Vue, HTML/CSS
Logic nghiệp vụ (Business Logic)	Logic cốt lõi	Chứa các quy tắc nghiệp vụ, logic xử lý dữ liệu và điều phối các tác vụ	Spring Boot, .NET Core, Node.js/Express
Dữ liệu (Data)	Lưu trữ và quản lý dữ liệu	Lưu trữ, quản lý, truy xuất và đảm bảo tính toàn vẹn của dữ liệu	SQL Server, PostgreSQL, MongoDB

Ưu điểm

- Khả năng tái sử dụng: Logic nghiệp vụ nằm ở tầng giữa, độc lập với tầng trình bày. Vì vậy, một logic nghiệp vụ có thể được sử dụng bởi nhiều giao diện khác nhau. Ví dụ: một API xử lý thanh toán có thể được gọi từ cả ứng dụng web và ứng dụng di động.

- Khả năng mở rộng: Đây là ưu điểm lớn nhất so với kiến trúc nguyên khối. Bạn có thể mở rộng độc lập theo từng tầng.

Nhược điểm:

- Phức tạp: Việc thiết lập và quản lý kiến trúc phân tán nhiều tầng phức tạp hơn so với kiến trúc nguyên khối

- Độ trễ: Việc truyền dữ liệu qua nhiều tầng (qua mạng) có thể làm tăng độ trễ (latency) so với việc gọi hàm cục bộ

- Chi phí: Việc duy trì nhiều máy chủ/dịch vụ vật lý hoặc máy ảo cho mỗi tầng sẽ tốn kém hơn

Khi nào thì nên sử dụng kiến trúc 3-tầng:

Khi ứng dụng của bạn phức tạp, dự kiến phát triển lớn trong tương lai, cần khả năng mở rộng cao và bảo mật dữ liệu được ưu tiên

5.4. Kiến trúc vi dịch vụ (Microservices Architecture)

Kiến trúc vi dịch vụ (còn gọi là kiến trúc phân tán) là mô hình trong đó hệ thống được chia thành nhiều dịch vụ nhỏ độc lập. Mỗi dịch vụ chạy trong tiến trình riêng, có thể có cơ sở dữ liệu riêng, và giao tiếp với nhau qua mạng (thường là API). Có thể hình dung kiến trúc vi dịch vụ giống như một khu phức hợp với nhiều tòa nhà chuyên biệt, kết nối bằng hệ thống đường xá riêng.

Ưu điểm

- Khả năng mở rộng cao: Mỗi dịch vụ có thể được mở rộng độc lập tùy theo nhu cầu.

- Triển khai độc lập: Các nhóm có thể phát triển, kiểm thử và triển khai dịch vụ của mình mà không ảnh hưởng đến các dịch vụ khác.

- Đa dạng công nghệ: Mỗi dịch vụ có thể được viết bằng ngôn ngữ lập trình và sử dụng cơ sở dữ liệu phù hợp nhất với chức năng của nó.

- Khả năng chịu lỗi tốt: Lỗi ở một dịch vụ không làm sập toàn bộ hệ thống.

Nhược điểm

- Độ phức tạp cao: Việc quản lý nhiều dịch vụ phân tán đòi hỏi công cụ và kỹ năng chuyên biệt.
- Khó khăn trong giao tiếp và đồng bộ dữ liệu: Giao tiếp qua mạng có độ trễ và có thể gặp lỗi.
- Chi phí vận hành cao hơn: Cần nhiều tài nguyên hơn để chạy và giám sát nhiều dịch vụ.

Khi nào nên sử dụng mô hình microservices:

- Quy mô hệ thống lớn: Khi mã nguồn quá đồ sộ và các chức năng nghiệp vụ trở nên phức tạp, khó quản lý dưới dạng đơn khối.
- Nhu cầu mở rộng độc lập: Khi cần tăng cường tài nguyên cho riêng từng dịch vụ cụ thể (như Thanh toán, Tìm kiếm) mà không ảnh hưởng đến phần còn lại.
- Tối ưu hóa tổ chức: Khi có nhiều đội ngũ phát triển riêng biệt, cần sự tự chủ trong việc chọn công nghệ và triển khai để không phụ thuộc lẫn nhau.
- Tính sẵn sàng và tốc độ: Khi cần cập nhật tính năng liên tục (CI/CD) và yêu cầu hệ thống không bị gián đoạn hoàn toàn nếu một thành phần nhỏ gặp sự cố.

5.5. So sánh và lựa chọn kiến trúc

Việc lựa chọn kiến trúc phù hợp phụ thuộc vào nhiều yếu tố:

- Quy mô và độ phức tạp của hệ thống: Hệ thống nhỏ có thể bắt đầu với kiến trúc nguyên khối và chuyển dần sang vi dịch vụ khi cần thiết.
- Yêu cầu về khả năng mở rộng: Nếu dự kiến hệ thống sẽ phục vụ lượng lớn người dùng và cần mở rộng nhanh chóng, kiến trúc N-tầng hoặc vi dịch vụ là lựa chọn phù hợp.
- Kinh nghiệm và quy mô của đội ngũ phát triển: Kiến trúc vi dịch vụ đòi hỏi đội ngũ có kinh nghiệm và quy mô đủ lớn để quản lý nhiều dịch vụ độc lập.
- Ngân sách và thời gian: Kiến trúc nguyên khối đơn giản hơn, chi phí phát triển và vận hành thấp hơn, phù hợp với các dự án có nguồn lực hạn chế.

CHƯƠNG 6. THIẾT KẾ CHI TIẾT

Sau khi đã lựa chọn được kiến trúc phù hợp cho hệ thống, bước tiếp theo trong quy trình thiết kế là thiết kế chi tiết. Thiết kế chi tiết là giai đoạn trung gian, nằm giữa thiết kế kiến trúc và giai đoạn lập trình. Nếu thiết kế kiến trúc là việc vẽ ra "bản quy hoạch" cho cả tòa nhà, thì thiết kế chi tiết chính là "bản vẽ kỹ thuật" mô tả cách lắp đặt từng đường điện, ống nước và vị trí nội thất trong từng căn phòng.

6.1. Mục tiêu của thiết kế chi tiết

Mục tiêu của thiết kế chi tiết là làm rõ cấu trúc bên trong của phần mềm, để người lập trình có thể viết mã mà không cần phải đặt câu hỏi về logic nghiệp vụ hay cấu trúc dữ liệu. Bản thiết kế chi tiết giúp:

- Giảm sai sót và sự mơ hồ khi lập trình: Lập trình viên có hướng dẫn rõ ràng để làm theo.
- Tăng khả năng bảo trì và mở rộng hệ thống: Cấu trúc được thiết kế tốt giúp dễ dàng thay đổi và nâng cấp sau này.
- Ước tính chính xác hơn thời gian và nguồn lực cần thiết: Khi đã có thiết kế chi tiết, việc ước lượng công sức lập trình sẽ chính xác hơn.

6.2. Nội dung của thiết kế chi tiết

Thiết kế chi tiết tập trung vào bốn nội dung chính, nhằm chuyển các ý tưởng của bản thiết kế kiến trúc thành các chỉ dẫn kỹ thuật cụ thể:

1. Thiết kế dữ liệu chi tiết

Đây là bước cụ thể hóa sơ đồ thực thể quan hệ (ERD) từ giai đoạn kiến trúc thành cấu trúc lưu trữ thực tế.

- Cơ sở dữ liệu: Xác định chính xác tên bảng, kiểu dữ liệu, độ dài trường dữ liệu, các ràng buộc (Not Null, Unique) và các khóa (Primary Key, Foreign Key).
- Cấu trúc dữ liệu trong bộ nhớ: Thiết kế các mảng (Array), danh sách (List), hay các cấu trúc dữ liệu phức tạp để xử lý dữ liệu tạm thời trong quá trình phần mềm chạy.

2. Thiết kế logic xử lý

Mô tả cách thức hoạt động bên trong của từng chức năng. Thay vì viết mã ngay, kiến trúc sư sẽ sử dụng công cụ trung gian để mô tả logic.

- Công cụ: Sử dụng lưu đồ (flowchart) hoặc mã giả (pseudo-code).
- Nội dung: Xác định các bước rẽ nhánh, vòng lặp, và cách lọc dữ liệu.

3. Thiết kế giao diện lập trình (API Design)

Đây là việc thiết kế các "hộp đồng" kết nối giữa các thành phần phần mềm (không phải giao diện người dùng).

- Định nghĩa API/Phương thức: Xác định tên hàm, danh sách tham số đầu vào (input) và kiểu dữ liệu trả về (output).
- Mục tiêu: Giúp các nhóm lập trình viên có thể làm việc song song. Nhóm A chỉ cần biết nhóm B cung cấp hàm `calculateTotal(items)` là có thể gọi để dùng, không cần quan tâm bên trong nhóm B viết gì.

4. Thiết kế phân cấp lớp (Class Design)

Dành cho các ngôn ngữ lập trình hướng đối tượng (OOP).

- Cấu trúc: Xác định các thuộc tính và phương thức cho từng lớp.
- Mối quan hệ: Mô tả các mối quan hệ kế thừa (Inheritance), bao đóng (Encapsulation) và đa hình (Polymorphism) để tối ưu hóa việc tái sử dụng mã nguồn.

6.3. Mẫu thiết kế (Design Patterns)

Trong thiết kế chi tiết, thay vì "phát minh lại bánh xe", các nhà phát triển sử dụng các Mẫu thiết kế. Đây là các giải pháp mẫu cho các vấn đề thiết kế phổ biến, đã được kiểm chứng và đúc kết qua nhiều năm kinh nghiệm của cộng đồng phát triển phần mềm.

Các mẫu thiết kế được phân thành ba nhóm chính:

- Mẫu khởi tạo (Creational): Ví dụ như Singleton (đảm bảo một lớp chỉ có một đối tượng duy nhất, dùng cho kết nối Database).

- Mẫu cấu trúc (Structural): Ví dụ như Adapter (giúp các thành phần không tương thích có thể làm việc với nhau).
- Mẫu hành vi (Behavioral): Ví dụ như Observer (tự động cập nhật giao diện khi dữ liệu thay đổi).

6.4. Kết quả đầu ra của thiết kế chi tiết

Kết thúc giai đoạn này, sản phẩm đầu ra là tài liệu thiết kế chi tiết (Detailed Design Document – DDD). Đây là "kim chỉ nam" cho lập trình viên. Một tài liệu DDD tốt là khi lập trình viên chỉ cần nhìn vào đó và viết mã, gần như không phải đưa ra quyết định về mặt logic hay cấu trúc nữa.

CHƯƠNG 7. CÔNG CỤ HỖ TRỢ PHÁT TRIỂN PHẦN MỀM

Trong quá trình phát triển phần mềm hiện đại, bên cạnh việc nắm vững các nguyên lý và quy trình, việc sử dụng thành thạo các công cụ hỗ trợ là yếu tố không thể thiếu để nâng cao hiệu quả và chất lượng công việc. Chương này sẽ giới thiệu hai công cụ quan trọng và được sử dụng rộng rãi nhất trong ngành công nghiệp phần mềm hiện nay: Git – hệ thống quản lý phiên bản phân tán, và Docker – nền tảng container hóa ứng dụng.

7.1. Hệ thống quản lý phiên bản

Phiên bản là gì?

Trong lập trình, phiên bản (version) là các bản khác nhau của tập tin, thư mục hoặc toàn bộ mã nguồn dự án. Trong quá trình làm việc, mã nguồn của dự án luôn thay đổi, vì vậy các lập trình viên luôn có nhu cầu xem lại hoặc khôi phục lại trạng thái của dự án tại một thời điểm nào đó trong quá khứ.

Hệ thống quản lý phiên bản là gì?

Hệ thống quản lý phiên bản (Version Control System – VCS) là một phần mềm giúp chúng ta lưu lại từng thay đổi của mã nguồn dự án, và giúp lấy lại được các phiên bản trước đó nếu cần. Một hệ thống quản lý phiên bản thường có các chức năng chính sau:

- Khôi phục lại trạng thái của dự án ở các thời điểm khác nhau trong quá khứ.
- Biết được ai đã thực hiện các thay đổi trên dự án, và đã thay đổi những gì.
- Dễ dàng khôi phục lại các nội dung mã nguồn bị xóa.
- Dễ dàng so sánh những thay đổi của dự án theo các mốc thời gian.

Dựa trên cách tổ chức và hoạt động, các hệ thống quản lý phiên bản được chia thành ba loại: hệ thống quản lý phiên bản cục bộ, hệ thống quản lý phiên bản tập trung, và hệ thống quản lý phiên bản phân tán.

Hệ thống quản lý phiên bản cục bộ

Đây là phương pháp đơn giản nhất, có thể thực hiện thủ công bằng cách lưu trữ các phiên bản khác nhau của thư mục dự án với các tên khác nhau, thường bao gồm tên, thời gian và dấu hiệu nhận diện. Tuy nhiên, khi dự án phát triển và có nhiều thay đổi, cách làm này trở nên khó theo dõi và quản lý, đặc biệt là không hỗ trợ làm việc nhóm.

Hệ thống quản lý phiên bản tập trung

Hệ thống quản lý phiên bản tập trung (Centralized Version Control Systems – CVCS) gồm một máy chủ chứa tất cả các phiên bản của dự án và các máy khách được phép truy cập. Các máy khách sẽ lấy phiên bản từ máy chủ về, thực hiện thay đổi và cập nhật lại lên máy chủ. Ví dụ: CVS, Subversion (SVN). Hạn chế chính của mô hình này là nếu máy chủ gặp sự cố, toàn bộ lịch sử phiên bản có thể bị mất và công việc của cả nhóm bị gián đoạn.

Hệ thống quản lý phiên bản phân tán

Hệ thống quản lý phiên bản phân tán (Distributed Version Control Systems – DVCS) khắc phục các hạn chế của CVCS. Với DVCS, mỗi máy client không chỉ sao chép phiên bản mới nhất mà còn sao chép toàn bộ kho chứa (repository), bao gồm cả lịch sử các phiên bản. Do đó, nếu máy chủ gặp trục trặc, vẫn có thể khôi phục toàn bộ kho chứa từ bất kỳ máy client nào. Ví dụ: Git, Mercurial, Darcs.

7.2. Git – Hệ thống quản lý phiên bản phân tán

Git là một hệ thống quản lý phiên bản phân tán được sử dụng rộng rãi nhất trong phát triển phần mềm hiện nay. Nó cho phép các nhóm lập trình viên theo dõi và quản lý các thay đổi trong mã nguồn của một dự án một cách hiệu quả.

Các tính năng chính của Git

- Theo dõi lịch sử thay đổi: Git lưu lại từng thay đổi nhỏ nhất của mã nguồn, giúp dễ dàng quay lại các phiên bản trước đó nếu cần.
- Cộng tác hiệu quả: Git cho phép nhiều người cùng làm việc trên một dự án cùng lúc, đồng thời hợp nhất các thay đổi một cách dễ dàng.
- Phân nhánh và hợp nhất: Git hỗ trợ tạo nhiều nhánh (branch) làm việc độc lập, giúp thử nghiệm các tính năng mới mà không ảnh hưởng đến phần còn lại của dự án.

- Bảo mật: Git lưu trữ các thay đổi dưới dạng các bản ghi (commit) không thể thay đổi, đảm bảo tính toàn vẹn của mã nguồn.

- Phân tán: Mỗi bản sao của một kho lưu trữ Git đều là một kho lưu trữ đầy đủ, cho phép làm việc không cần kết nối mạng và đồng bộ hóa sau đó.

Nhúng Git vào dự án

Để sử dụng Git quản lý mã nguồn của một dự án, cần thực hiện lệnh git init trong thư mục gốc của dự án. Lệnh này sẽ tạo ra một thư mục ẩn có tên là .git – đây chính là kho chứa (repository) mà Git sử dụng để lưu trữ toàn bộ lịch sử thay đổi của dự án.

Cấu hình định danh người dùng

Trong quá trình phát triển phần mềm, đặc biệt là khi làm việc nhóm, việc theo dõi và quản lý các thay đổi là vô cùng quan trọng. Git yêu cầu mỗi commit đều phải gắn liền với thông tin người thực hiện, cho phép:

- Xác định tác giả: dễ dàng biết được ai là người viết hoặc chỉnh sửa một đoạn mã cụ thể.
- Theo dõi đóng góp: nắm bắt được lịch sử đóng góp của từng thành viên trong dự án.
- Phân công trách nhiệm: quy trách nhiệm cho từng thành viên đối với những thay đổi mà họ thực hiện.

Git cho phép cấu hình ở ba phạm vi khác nhau với độ ưu tiên từ cao đến thấp: local (áp dụng cho một kho lưu trữ cụ thể), global (áp dụng cho tài khoản người dùng hiện tại), và system (áp dụng cho tất cả người dùng trên hệ thống).

Trước khi làm việc với Git, cần cấu hình danh tính người dùng:

```
git config --global user.name "Tên của bạn"
```

```
git config --global user.email "email@example.com"
```

7.3. Các khu vực làm việc của Git

Khi làm việc với Git, các lập trình viên sẽ thao tác với ba khu vực chính:

1. Thư mục làm việc (Working Directory)

Đây là nơi bạn chỉnh sửa tập tin trực tiếp trên máy tính. Nó đại diện cho trạng thái hiện tại của mã nguồn, bao gồm cả các tập tin đã được Git theo dõi (tracked) và chưa theo dõi (untracked). Thư mục làm việc là bản sao "sống" của kho chứa tại một thời điểm nhất định.

2. Khu vực tổ chức tạm (Staging Area)

Còn gọi là Khu tạm hay Index, đây là vùng trung gian lưu trữ các thay đổi mà bạn đã chọn để commit. Dùng lệnh git add để thêm tập tin từ Thư mục làm việc vào Khu tạm. Khu tạm cho phép bạn chọn lọc và tổ chức các thay đổi muốn đưa vào commit tiếp theo, thay vì commit tất cả mọi thứ.

3. Kho chứa (Repository)

Đây là nơi Git lưu trữ tất cả các tập tin và lịch sử thay đổi của dự án. Thư mục .git trong dự án chính là Kho chứa. Dùng lệnh git commit để chuyển các thay đổi từ Khu tạm vào Kho chứa, tạo ra một phiên bản mới trong lịch sử phát triển.

Quy trình làm việc cơ bản với Git diễn ra theo thứ tự: chỉnh sửa mã nguồn ở Thư mục làm việc → chuẩn bị nội dung commit ở Khu tạm (dùng git add) → lưu trữ mã nguồn và lịch sử ở Kho chứa (dùng git commit).

Gitignore

.gitignore là một cơ chế trong Git cho phép chỉ định các tập tin hoặc thư mục mà Git sẽ bỏ qua (không theo dõi). Điều này rất hữu ích để loại bỏ các tập tin không cần thiết khỏi lịch sử phiên bản, như tập tin tạm, thư viện phụ thuộc (node_modules), tập tin cấu hình môi trường (.env), hay tập tin nhật ký (*.log)

7.4. Nguyên tắc làm việc của Git

Cơ chế "chụp ảnh" (Snapshot)

Khác với hầu hết các hệ thống quản lý phiên bản khác (như CVS, SVN) lưu trữ các thay đổi dưới dạng delta (sự khác biệt giữa các phiên bản), Git sử dụng cơ chế "chụp ảnh" (snapshot). Để lưu lại một phiên bản của dự án, Git chụp ảnh toàn bộ các tập tin. Tập tin nào có thay đổi thì tạo ra một ảnh mới; tập tin nào không thay đổi thì chỉ tạo

một liên kết trở về phiên bản trước của nó. Mỗi "ảnh chụp" tương đương với một commit trong hệ thống Git.

Phần lớn thao tác được thực hiện cục bộ

Khi làm việc với Git, hầu hết các thông tin và tài nguyên đều có sẵn trên máy cục bộ. Bạn có thể xem lịch sử phiên bản, tạo commit, chuyển đổi giữa các nhánh mà không cần kết nối mạng. Chỉ khi cần chia sẻ thay đổi với người khác, bạn mới cần kết nối để đẩy (push) lên server hoặc kéo (pull) về từ server.

Đảm bảo tính toàn vẹn dữ liệu

Mọi đối tượng trong Git đều được tạo mã kiểm tra (checksum) bằng thuật toán SHA-1 trước khi lưu vào cơ sở dữ liệu. Git sử dụng mã SHA-1 này để đặt tên và thao tác với tập tin. Nếu nội dung tập tin bị thay đổi, Git sẽ tính lại mã SHA-1 và phát hiện ra sự thay đổi. Cơ chế này đảm bảo dữ liệu do Git quản lý luôn có tính toàn vẹn, không thể bị thay đổi mà Git không biết.

Ba trạng thái của tập tin

Khi một tập tin đã được Git quản lý, nó có thể ở một trong ba trạng thái:

- **Modified:** Tập tin đã bị thay đổi nhưng chưa được lưu vào Kho chứa (chưa commit).
- **Staged:** Tập tin đã được đánh dấu và đưa vào Khu tạm, sẵn sàng cho lần commit tiếp theo.
- **Committed:** Tập tin đã được lưu an toàn vào Kho chứa.

Các quy trình làm việc nhóm (Git Workflow) phổ biến:

Git Flow: Sử dụng các nhánh cố định (main, develop, feature, release, hotfix) với quy tắc nghiêm ngặt. Phù hợp với dự án có chu kỳ release rõ ràng (phiên bản định kỳ). Phức tạp hơn nhưng kiểm soát tốt.

GitHub Flow: Đơn giản hơn, chỉ có nhánh main và các nhánh tính năng. Mỗi tính năng là một Pull Request được review trước khi merge. Phù hợp với triển khai liên tục (Continuous Deployment).

Trunk-Based Development: Tất cả commit trực tiếp vào nhánh chính thường xuyên. Đòi hỏi kiểm thử tự động rất mạnh.

Nguyên tắc commit tốt:

- Mỗi commit chỉ nên chứa một thay đổi logic có liên quan
- Message commit rõ ràng, mô tả đúng nội dung thay đổi (nên theo chuẩn Conventional Commits: feat:, fix:, docs:, refactor:...)

- Không commit code bị lỗi hoặc code chưa kiểm thử lên nhánh chính
- Thực hiện code review qua Pull Request / Merge Request trước khi hợp nhất

7.5. Docker – Nền tảng container hóa ứng dụng

Trong phát triển phần mềm, một vấn đề thường gặp là ứng dụng chạy tốt trên máy của lập trình viên nhưng gặp lỗi khi triển khai lên máy chủ hoặc máy của đồng nghiệp. Nguyên nhân thường là do sự khác biệt về môi trường: phiên bản hệ điều hành, thư viện, công cụ phụ thuộc. Docker ra đời để giải quyết vấn đề này.

Docker là gì?

Docker là một nền tảng giúp "đóng gói" ứng dụng cùng với thư viện và môi trường cần thiết thành một Docker Image, sau đó có thể "chuyển giao" Image này tới bất kỳ máy tính nào khác và "khởi chạy" để tạo thành các Container. Tại máy người nhận, họ chỉ cần chạy Image, ứng dụng sẽ hoạt động ngay lập tức vì môi trường bên trong đã được chuẩn bị sẵn, không còn nỗi lo "máy em chạy được nhưng máy anh thì không".

Lợi ích của Docker

- Giúp mã nguồn chạy ổn định, cho ra kết quả giống nhau khi chạy trên các máy khác nhau.
- Có thể chạy ứng dụng với các phiên bản ngôn ngữ khác nhau (ví dụ Python 2 và Python 3) trên cùng một máy mà không sợ xung đột.
- Thiết lập môi trường và chạy ứng dụng chỉ trong vài giây thay vì phải cài đặt, thiết lập thủ công mất nhiều giờ.

So sánh Docker Container với Virtual Machine (VM):

Máy ảo ảo hóa toàn bộ phần cứng, bao gồm cả hệ điều hành khách riêng biệt, do đó nặng nề và khởi động chậm. Ngược lại, container dùng chung hạt nhân với máy chủ, chỉ cô lập ở mức tiến trình, nhẹ hơn và khởi động nhanh hơn rất nhiều.

Các thành phần chính của Docker

Docker gồm ba thành phần chính:

- **Docker Engine:** Là ứng dụng mã nguồn mở theo mô hình client-server, dùng để khởi tạo, vận hành và quản lý các Docker Image và Docker Container. Docker Engine gồm Docker Daemon (người thực hiện), REST API (hệ thống truyền tin) và Docker CLI (người điều khiển).

- **Docker Image:** Là một bản thiết kế hoặc khuôn mẫu đóng băng của một môi trường máy tính, chứa tất cả những thứ cần thiết để một ứng dụng có thể chạy được (mã nguồn, thư viện, công cụ hệ thống, cấu hình). Image có tính bất biến và được xây dựng từ nhiều lớp xếp chồng lên nhau.

- **Docker Container:** Là thực thể sống đang hoạt động được tạo ra từ Docker Image. Mỗi Container chạy trong một không gian biệt lập, có thể được tạo ra, khởi động, dừng lại hoặc xóa đi bất cứ lúc nào mà không ảnh hưởng đến Image gốc.

7.6. Docker Image và Docker Container

Hai khái niệm cốt lõi của Docker cần nắm vững là Image và Container.

Docker Image là một bản thiết kế (blueprint) đóng băng của một môi trường máy tính, chứa tất cả những gì cần thiết để ứng dụng hoạt động: mã nguồn, thư viện, công cụ hệ thống và các thiết lập cấu hình. Image có tính bất biến (immutable) – một khi đã được tạo ra, nội dung của nó không thể thay đổi. Mỗi image được xây dựng từ nhiều lớp (layer) xếp chồng lên nhau, mỗi lớp tương ứng với một lệnh trong tập tin Dockerfile. Cơ chế lớp giúp tận dụng bộ nhớ đệm (cache) để tăng tốc độ xây dựng image.

Docker Container là một thực thể đang hoạt động được tạo ra từ một image. Có thể hình dung: nếu image là bản vẽ chi tiết của một chiếc thùng container đã được niêm

phong trên bến cảng, thì container chính là chiếc thùng đó khi được mở ra và mọi máy móc bên trong đang vận hành. Từ một image duy nhất, ta có thể khởi tạo nhiều container chạy song song và hoàn toàn độc lập với nhau. Mỗi container là một không gian biệt lập, có thể được tạo ra, khởi động, dừng lại hoặc xóa bỏ bất cứ lúc nào mà không làm ảnh hưởng đến image gốc.

Mối quan hệ giữa image và container tương tự như mối quan hệ giữa lớp (class) và đối tượng (object) trong lập trình hướng đối tượng: image là khuôn mẫu, container là thể hiện cụ thể đang chạy.

Các lệnh Docker cơ bản thường dùng:

`docker pull <image>`: tải image từ registry về máy.

`docker build -t <tên> .`: xây dựng image từ Dockerfile trong thư mục hiện tại.

`docker run <image>`: tạo và khởi chạy container từ image.

`docker ps`: liệt kê các container đang chạy.

`docker stop <container>`: dừng container.

`docker images`: xem danh sách image có trên máy.

`docker push <image>`: đẩy image lên registry.

7.7. Dockerfile – Công thức xây dựng Image

Để tạo ra một Docker Image, ta viết một tập tin văn bản có tên là Dockerfile. Đây là bản hướng dẫn chứa một loạt các lệnh để Docker tự động xây dựng image. Mỗi lệnh trong Dockerfile sẽ tạo ra một lớp mới trong image. Nhờ cơ chế cache theo lớp, Docker chỉ thực hiện lại những lệnh có thay đổi so với lần build trước, giúp tăng tốc quá trình build đáng kể.

Các chỉ thị phổ biến trong Dockerfile:

FROM: chỉ định image gốc (base image) để bắt đầu xây dựng.

WORKDIR: thiết lập thư mục làm việc mặc định bên trong container.

COPY / ADD: sao chép tập tin hoặc thư mục từ máy chủ vào image.

RUN: thực thi các lệnh trong quá trình build (ví dụ: cài đặt thư viện, biên dịch mã nguồn).

ENV: đặt biến môi trường.

EXPOSE: khai báo cổng mạng mà container sẽ lắng nghe khi chạy.

CMD / ENTRYPOINT: chỉ định lệnh sẽ được thực thi khi container khởi động.

7.8. Quy trình Build – Ship - Run

Quy trình làm việc với Docker bao gồm ba bước cơ bản:

1. **Build (Xây dựng):** Viết tập tin Dockerfile (bản hướng dẫn) và chạy lệnh docker build để tạo Image. Docker Engine sẽ đọc bản hướng dẫn này để gom hệ điều hành, thư viện và mã nguồn lại thành một tập tin duy nhất.

2. **Ship (Chuyển giao):** Đưa Image tới máy tính khác (máy đồng nghiệp hoặc máy chủ).

3. **Run (Khởi chạy):** Chạy Image để tạo Container, ứng dụng sẽ hoạt động trong môi trường đã được đóng gói sẵn.

Ví dụ thực hành: Đóng gói ứng dụng web với Docker

Giả sử cần đóng gói một ứng dụng web đơn giản gồm một tập tin HTML chạy trên web server Nginx:

Bước 1: Tạo thư mục dự án và tập tin index.html với nội dung hiển thị lời chào.

Bước 2: Tạo tập tin Dockerfile với nội dung:

```
# Chọn hệ điều hành rút gọn có sẵn máy chủ web Nginx
```

```
FROM nginx:alpine
```

```
# Chép mã nguồn từ máy tính vào đúng vị trí trong Image
```

```
COPY index.html /usr/share/nginx/html/index.html
```

Bước 3: Chạy lệnh build để tạo Image:

```
Docker build -t hello-docker-image:v1 .
```

Sau khi build thành công, Image hello-docker-image sẽ xuất hiện trong Docker Desktop và có thể chạy container bằng lệnh:

```
docker run -d -p 8080:80 hello-docker:v1
```

Lúc này, mở trình duyệt và truy cập <http://localhost:8080> sẽ thấy dòng chữ "Hello Docker".

Một số thực hành tốt khi viết Dockerfile:

- Đặt các lệnh ít thay đổi (như cài đặt thư viện) lên trước để tận dụng cache, giảm thời gian build.
- Sử dụng `.dockerignore` để loại bỏ những file không cần thiết (ví dụ `node_modules`, `.git`) khỏi ngữ cảnh build.
- Áp dụng multi-stage build để tách biệt môi trường build và môi trường chạy, giúp giảm đáng kể kích thước image cuối cùng.

7.9. Docker Compose và tích hợp CI/CD

Khi ứng dụng phát triển phức tạp hơn, thường bao gồm nhiều dịch vụ phối hợp (ví dụ: web server, cơ sở dữ liệu, bộ nhớ đệm, hàng đợi tin nhắn). Việc quản lý từng container riêng lẻ trở nên cồng kềnh. Docker Compose là công cụ cho phép định nghĩa và chạy đồng thời nhiều container thông qua một file cấu hình duy nhất, thường là `docker-compose.yml`. Chỉ với một lệnh `docker-compose up`, toàn bộ hệ thống đa dịch vụ sẽ được khởi tạo.

Docker cũng tích hợp sâu vào quy trình CI/CD (Tích hợp liên tục / Triển khai liên tục). Một pipeline CI/CD điển hình với Docker có thể được mô tả như sau:

1. Lập trình viên đẩy mã nguồn lên kho Git (GitHub, GitLab).
2. Hệ thống CI (Jenkins, GitHub Actions, GitLab CI) tự động kích hoạt pipeline khi phát hiện thay đổi.
3. Pipeline thực thi các bài kiểm thử tự động (unit test, integration test).
4. Nếu tất cả kiểm thử đều đạt, CI server tiến hành build Docker Image mới từ Dockerfile.

5. Image sau khi build được đẩy lên Docker Registry (Docker Hub, AWS ECR, Google Container Registry...).

6. Hệ thống CD tự động triển khai image mới lên môi trường staging hoặc production, thay thế container cũ bằng container mới một cách trơn tru.

7. Các công cụ giám sát (như Grafana, Prometheus) theo dõi tình trạng hệ thống sau triển khai và cảnh báo nếu có lỗi.

Khi kết hợp Git + Docker + CI/CD, mỗi lần lập trình viên đẩy mã nguồn mới, toàn bộ quy trình từ kiểm thử, đóng gói đến triển khai diễn ra hoàn toàn tự động. Điều này không chỉ giảm thiểu sai sót do thao tác thủ công mà còn rút ngắn đáng kể thời gian đưa tính năng mới đến tay người dùng, nâng cao chất lượng và độ tin cậy của sản phẩm.

CHƯƠNG 8. BÀI TẬP VÀ KẾT QUẢ THỰC HÀNH

Để củng cố kiến thức lý thuyết đã trình bày ở các chương trước, em đã thực hiện đầy đủ các bài tập sau mỗi buổi học và tiến hành thực hành trên máy với các công cụ Git, Docker. Nội dung dưới đây tổng hợp các bài tập tiêu biểu và kết quả thực hành đã đạt được, kèm theo những minh chứng cụ thể.

8.1. Bài tập và thực hành môn Công nghệ phần mềm

8.1.1. Bài tập CNPM 1 – Giới thiệu

Bài tập 1. Bạn sẽ lựa chọn theo Công nghệ phần mềm hay Khoa học máy tính? Tại sao?

Bài làm:

Em lựa chọn **Công nghệ phần mềm**.

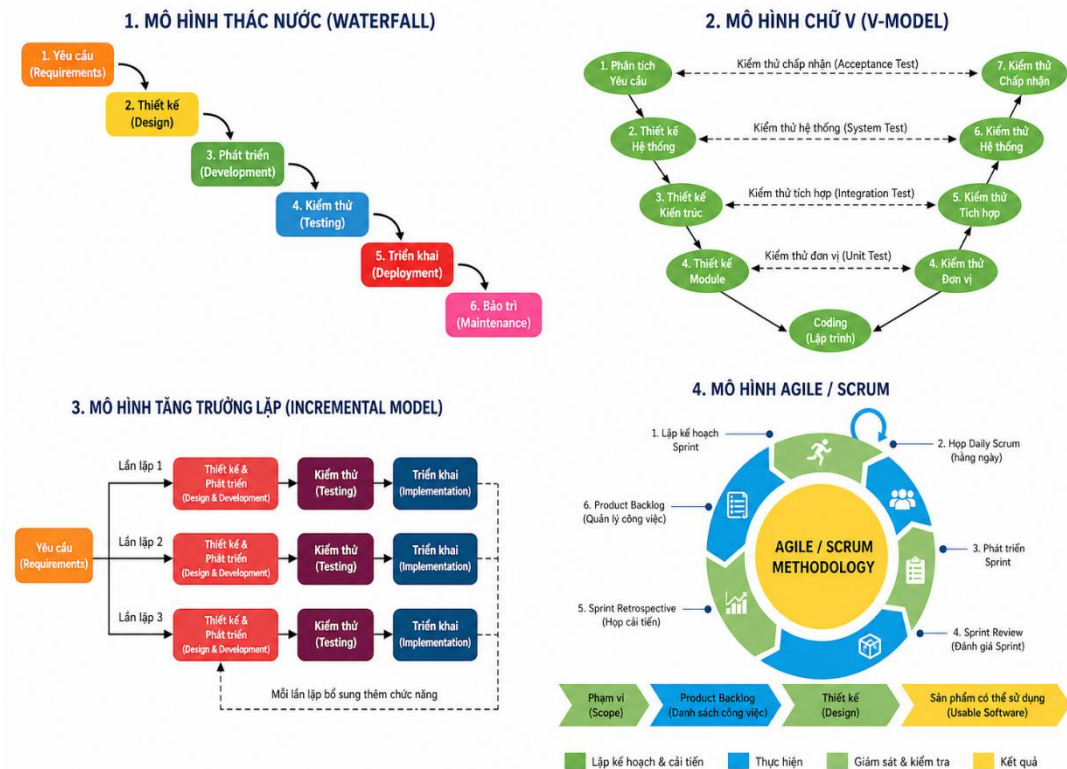
Lý do em chọn Công nghệ phần mềm là vì đây là ngành học tập trung vào việc **phân tích, thiết kế, xây dựng, kiểm thử và bảo trì các hệ thống phần mềm** phục vụ trực tiếp cho nhu cầu thực tế. Ngành này giúp em rèn luyện tư duy logic, kỹ năng làm việc nhóm, quản lý dự án và áp dụng kiến thức vào các sản phẩm cụ thể.

Bên cạnh đó, Công nghệ phần mềm có **tính ứng dụng cao**, cơ hội việc làm rộng mở với nhiều vị trí như lập trình viên, kiểm thử phần mềm, phân tích hệ thống hay quản lý dự án. Em cảm thấy ngành học này **phù hợp với định hướng nghề nghiệp**, sở thích lập trình và mong muốn tạo ra các sản phẩm phần mềm hữu ích trong tương lai.

Vì những lý do trên, em quyết định lựa chọn **Công nghệ phần mềm** thay vì Khoa học máy tính.

8.1.2. Bài tập CNPM 2 – Các mô hình phát triển phần mềm

Bài tập 2a. Vẽ lại sơ đồ các mô hình: Thác nước, chữ V, Tăng trưởng lặp và Agile/Scrum.



Bài tập 2b. Mô tả công việc của mỗi công đoạn, công cụ sử dụng của mô hình Thác nước.

Giai đoạn	Công việc chính	Công cụ sử dụng
[1] Lấy và phân tích yêu cầu	Làm việc trực tiếp với khách hàng để xác định các tính năng, giao diện và hiệu suất; sử dụng các kỹ thuật như phỏng vấn, khảo sát hoặc viết user stories; phân loại yêu cầu thành nhóm chức năng và phi chức năng; xây dựng tài liệu SRS	Word, Google Docs, Confluence
[2] Thiết kế	Chuyển các yêu cầu thành bản vẽ kỹ thuật; thiết kế	Enterprise Architect, Visual Paradigm,

	kiến trúc hệ thống, cơ sở dữ liệu, thuật toán và giao diện người dùng	Lucidchart (vẽ UML), Figma (thiết kế UI/UX)
[3] Lập trình	Viết mã dựa trên bản thiết kế đã có; tuân thủ các tiêu chuẩn lập trình (SOLID, DRY, KISS); viết unit test	VS Code, Cursor, IntelliJ, Git
[4] Kiểm thử	Tìm và sửa lỗi; đảm bảo phần mềm chạy đúng, ổn định và đáp ứng đúng yêu cầu ban đầu của khách hàng; thực hiện kiểm thử đơn vị, tích hợp, hệ thống và chấp nhận	Selenium, JMeter, Postman
[5] Triển khai	Cài đặt phần mềm vào môi trường thực tế; bàn giao cho khách hàng và hướng dẫn họ cách sử dụng sản phẩm; thiết lập CI/CD	Docker, Jenkins, GitHub Actions
[6] Vận hành và Bảo trì	Theo dõi hệ thống, sửa các lỗi phát sinh sau khi dùng; cập nhật tính năng mới hoặc nâng cấp để tương thích với công nghệ mới; tái cấu trúc mã nguồn	Grafana, Prometheus

8.1.3. Bài tập CNPM 3 – Quản lý dự án phần mềm

Bài tập 3a. Dự án của cá nhân

Yêu cầu: Mỗi cá nhân viết nội dung cho mỗi giai đoạn của dự án (theo mẫu Vòng đời của dự án), viết trên Microsoft Word, tạo một thư mục của cá nhân trên Google Drive để chứa các kết quả làm việc, và chia sẻ thư mục cho giáo viên (khi có yêu cầu).

Bài làm:

Vòng đời dự án "Hệ thống bán sách trực tuyến" bao gồm:

1. Khởi xướng: Xác định mục tiêu dự án, các bên liên quan (khách hàng là công ty sách ABC, nhà tài trợ là ban giám đốc, nhóm phát triển gồm 5 thành viên).
2. Lập kế hoạch: Xây dựng WBS sơ bộ, ước tính thời gian 12 tuần, ngân sách dự kiến 200 triệu đồng.
3. Triển khai thực hiện: Phân công công việc cho từng thành viên, tổ chức họp Scrum hàng ngày.
4. Giám sát và kiểm soát: Theo dõi tiến độ bằng biểu đồ Gantt, kiểm soát chất lượng qua các buổi review Sprint.
5. Kết thúc: Nghiệm thu sản phẩm, bàn giao tài liệu hướng dẫn sử dụng, tổ chức họp rút kinh nghiệm.

Bài tập 3b. Viết tôn chỉ dự án

Yêu cầu: Tham khảo mẫu, viết bản tôn chỉ dự án của cá nhân/nhóm, chép kết quả lên thư mục dự án của nhóm trên Google Drive

Bài làm:

Tôn chỉ dự án (Project Charter) cho dự án "Hệ thống bán sách trực tuyến" theo mẫu, bao gồm các nội dung chính:

- Tên dự án: Hệ thống bán sách trực tuyến – BookStore Online
- Mục tiêu dự án: Xây dựng website thương mại điện tử cho phép người dùng tìm kiếm, đặt mua sách trực tuyến và theo dõi đơn hàng

- Phạm vi dự án: Phát triển các module: quản lý người dùng, danh mục sản phẩm, giỏ hàng, thanh toán, quản lý đơn hàng
- Thời gian dự kiến: 12 tuần (từ 01/03/2026 đến 24/05/2026)
- Ngân sách dự kiến: 200.000.000 VNĐ
- Nhà tài trợ: Công ty ABC
- Quản lý dự án: Nguyễn Văn A
- Các bên liên quan chính: Khách hàng (công ty ABC), nhóm phát triển (5 thành viên), người dùng cuối

Bài tập 3c. Tự đọc thêm, cài đặt và trải nghiệm với Jira, Trello.

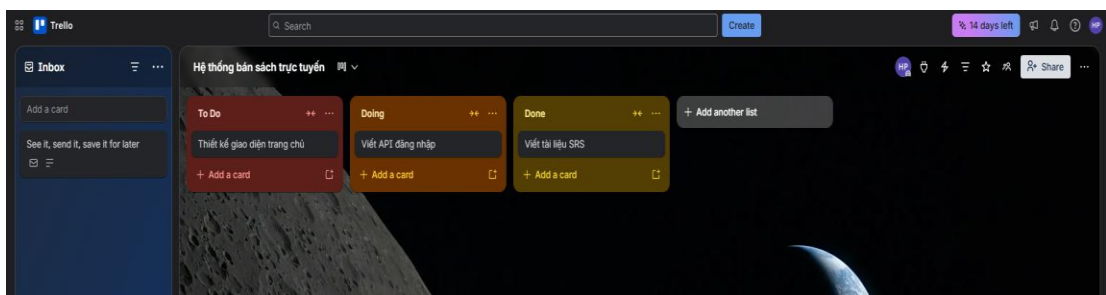
Giới thiệu

Jira và Trello là hai công cụ quản lý dự án phổ biến nhất hiện nay, đều do công ty Atlassian phát triển. Trong thực tế, hầu hết các công ty phần mềm đều sử dụng một trong hai công cụ này để theo dõi tiến độ, phân công công việc và quản lý Sprint.

Trello

Trello là công cụ quản lý công việc trực quan theo phương pháp Kanban. Giao diện chính gồm các thành phần:

- Board (Bảng): Đại diện cho một dự án
- List (Danh sách): Các cột thể hiện trạng thái công việc, thường là: To Do / In Progress / Done
- Card (Thẻ): Mỗi thẻ là một công việc cụ thể, có thể gán người thực hiện, deadline, checklist, nhãn màu



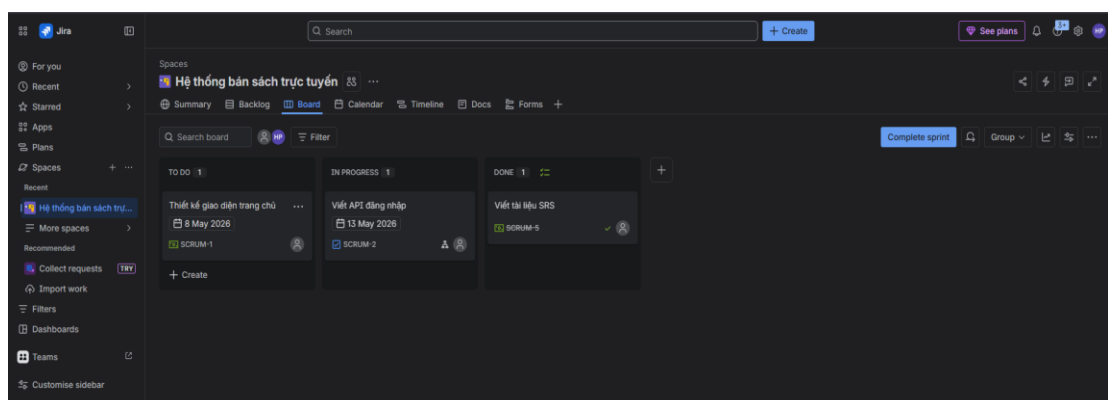
Sau khi cài đặt và sử dụng Trello cho dự án "Hệ thống bán sách trực tuyến", em đã tạo Board với các công việc phân chia theo 3 trạng thái. Trello giúp cả nhóm nhìn thấy toàn bộ tiến độ dự án chỉ trong một màn hình, dễ cập nhật và theo dõi hơn so với quản lý bằng file Word hay Excel.

Nhận xét: Trello đơn giản, dễ dùng, phù hợp với nhóm nhỏ hoặc người mới bắt đầu làm quen với quản lý dự án theo phương pháp Kanban.

Jira

Jira là công cụ quản lý dự án chuyên nghiệp hơn, được thiết kế đặc biệt cho các nhóm phát triển phần mềm theo phương pháp Agile/Scrum. Các khái niệm chính trong Jira:

- Project: Dự án phần mềm cần quản lý
- Epic: Tính năng lớn, bao gồm nhiều Story nhỏ hơn
- User Story: Yêu cầu người dùng viết theo cấu trúc "Là [ai], tôi muốn [làm gì], để [đạt mục đích gì]"
- Task / Sub-task: Công việc cụ thể cần thực hiện
- Sprint: Chu kỳ phát triển 1–4 tuần
- Backlog: Danh sách toàn bộ công việc chưa thực hiện
- Board: Bảng theo dõi tiến độ Sprint hiện tại



Sau khi cài đặt và trải nghiệm Jira, em đã tạo Project Scrum cho dự án "Hệ thống bán sách trực tuyến" em thấy Jira phức tạp hơn Trello nhưng mạnh mẽ hơn nhiều, phù

hợp với nhóm phát triển chuyên nghiệp. Jira tích hợp trực tiếp với GitHub/GitLab, cho phép liên kết commit với Task, rất tiện lợi trong thực tế làm việc.

8.1.4. Bài tập CNPM 4 – Lấy và phân tích yêu cầu

Bài tập 4a. Viết URS cho dự án của bạn

Yêu cầu: Dựa vào Tài liệu đặc tả yêu cầu người dùng (User Requirement Specification - URS), viết URS cho dự án của nhóm.

Bài làm:

Tài liệu URS cho dự án "Hệ thống bán sách trực tuyến" bao gồm:

1. Giới thiệu: Tài liệu này trình bày các Yêu cầu người dùng cho dự án phát triển hệ thống Bán sách trực tuyến. Hệ thống được xây dựng nhằm cung cấp một nền tảng thương mại điện tử, giúp người dùng dễ dàng tìm kiếm, mua sắm sách và quản lý các đơn hàng của mình. Mục đích của tài liệu là làm rõ những mong đợi và kỳ vọng của người dùng cuối, là cơ sở để đội ngũ phát triển và các bên liên quan cùng hiểu và thực hiện dự án.

2.1. Yêu cầu Chức năng (Functional Requirements - FR)

2. Các loại yêu cầu:

Yêu cầu liên quan đến Người dùng và Tài khoản:

- FR-1.1: Hệ thống phải cho phép người dùng mới đăng ký tài khoản bằng cách cung cấp email, tên đầy đủ và mật khẩu.

- FR-1.2: Hệ thống phải xác thực thông tin đăng nhập của người dùng đã có tài khoản.

- FR-1.3: Hệ thống phải có chức năng "Quên mật khẩu", cho phép người dùng đặt lại mật khẩu qua email.

- FR-1.4: Người dùng phải có khả năng cập nhật thông tin cá nhân của họ, bao gồm tên, địa chỉ và số điện thoại.

- FR-1.5: Người dùng phải có thể xem lịch sử mua hàng, bao gồm các đơn hàng đã đặt, trạng thái đơn hàng và chi tiết sản phẩm đã mua.

Yêu cầu liên quan đến Sản phẩm và Tìm kiếm:

- FR-2.1: Hệ thống phải cho phép người dùng tìm kiếm sách theo tiêu đề, tác giả, hoặc nhà xuất bản.

(Tài liệu đầy đủ tiếp tục với tất cả các yêu cầu chức năng và phi chức năng)

Bài tập 4b. Từ URS trên chuyển thành SRS

Yêu cầu: Chuyển từ URS thành SRS (Software Requirements Specification).

Bài làm:

Chuyển đổi tài liệu URS thành SRS cho dự án "Hệ thống bán sách trực tuyến". Tài liệu SRS cụ thể hóa các yêu cầu kỹ thuật, phục vụ cho việc thiết kế, lập trình và kiểm thử. Tài liệu bao gồm:

1. Giới thiệu: Mô tả mục đích, phạm vi, định nghĩa thuật ngữ
2. Mô tả tổng quan: Góc nhìn sản phẩm, các đối tượng người dùng, môi trường hoạt động
3. Yêu cầu chức năng chi tiết: Mỗi yêu cầu được mô tả với Actor, Pre-condition, Post-condition, Main Flow, Alternative Flow
4. Yêu cầu phi chức năng: Hiệu suất (phản hồi < 2 giây), bảo mật (mã hóa dữ liệu), khả năng mở rộng (hỗ trợ 10.000 người dùng đồng thời)
5. Yêu cầu giao diện: Wireframe các màn hình chính
6. Phụ lục: Các quy tắc nghiệp vụ (thuế VAT 10%, phí vận chuyển, giới hạn đơn hàng)

8.1.5. Bài tập CNPM 5 – Thiết kế kiến trúc phần mềm

Bài tập 5a. Dựa trên URS ứng dụng của bạn (nhóm), hãy phân tích và thiết kế theo kiến trúc 3-tầng.

Bài làm:

Dựa trên URS của dự án "Hệ thống bán sách trực tuyến", kiến trúc 3-tầng như sau:

Tầng Trình bày (Presentation Tier):

- Công nghệ: ReactJS kết hợp với Bootstrap 5
- Nhiệm vụ: Hiển thị giao diện người dùng (trang chủ, danh sách sản phẩm, giỏ hàng, thanh toán), nhận yêu cầu từ người dùng và gửi HTTP Request đến tầng Logic nghiệp vụ
- Thành phần chính: Các component React (Header, ProductList, Cart, Checkout), sử dụng React Router để điều hướng, Axios để gọi API

Tầng Logic nghiệp vụ (Business Logic Tier):

- Công nghệ: Spring Boot (Java) triển khai RESTful API
- Nhiệm vụ: Xử lý các quy tắc nghiệp vụ (xác thực người dùng, tính toán giỏ hàng, kiểm tra tồn kho, xử lý đơn hàng), điều phối giữa các service
- Thành phần chính: AuthService, ProductService, CartService, OrderService, PaymentService
- API Gateway: Sử dụng Spring Cloud Gateway để định tuyến request

Tầng Dữ liệu (Data Tier):

- Công nghệ: PostgreSQL cho dữ liệu quan hệ, Redis cho cache và session
- Nhiệm vụ: Lưu trữ, truy xuất và đảm bảo tính toàn vẹn của dữ liệu
- Bảng dữ liệu: users, products, categories, orders, order_items, carts, reviews

Bài tập 5b. Dựa trên URS ứng dụng của bạn (nhóm), hãy phân tích và thiết kế theo kiến trúc Microservices.

Bài làm:

Em đã phân tích và đề xuất thiết kế kiến trúc Microservices cho hệ thống, chia thành các service độc lập:

1. User Service: Quản lý đăng ký, đăng nhập, thông tin người dùng (Database riêng: PostgreSQL)

2. Product Service: Quản lý danh mục và thông tin sản phẩm (Database riêng: PostgreSQL)

3. Cart Service: Quản lý giỏ hàng (Database riêng: Redis)

4. Order Service: Xử lý đơn hàng (Database riêng: PostgreSQL)

5. Payment Service: Xử lý thanh toán, tích hợp cổng thanh toán (Database riêng: PostgreSQL)

6. Notification Service: Gửi email xác nhận, thông báo (sử dụng Message Queue RabbitMQ)

7. API Gateway: Spring Cloud Gateway để định tuyến request từ client đến các service

8. Service Discovery: Eureka Server để đăng ký và khám phá các service

9. Centralized Logging & Monitoring: ELK Stack (Elasticsearch, Logstash, Kibana) và Prometheus + Grafana

8.2. Thực hành Git

8.2.1. Git thực hành 1 – Cấu hình định danh người dùng

Bài tập 1.1 Bài tập tình huống “Quản lý cấu hình Git cho nhiều dự án”

Giả sử bạn là Nguyễn Văn Tèo, một lập trình viên đang làm việc tại công ty TeoTech. Ngoài công việc chính, bạn còn tham gia phát triển một dự án mang tên VienVong (viễn vông) cùng với một nhóm bạn vào thời gian rảnh.

Để quản lý mã nguồn hiệu quả, bạn sử dụng Git cho cả hai dự án. Tuy nhiên, bạn muốn đảm bảo rằng:

- Khi làm việc tại công ty TeoTech, các commit của bạn sẽ được gắn với thông tin định danh của công ty (tên và email công ty).

- Khi làm việc với dự án "viễn vông", các commit của bạn sẽ được gắn với thông tin định danh cá nhân.

Bài làm:

1. Tạo thư mục 2 dự án

Mở terminal hoặc git bash

`mkdir TeoTechProject`

`mkdir VienVongProject`

2. Khởi tạo Git repository cho từng dự án

2.1. Dự án công ty TeoTechProject

`cd TeoTechProject → git init`

```
D:\>mkdir TeoTechProject

D:\>cd TeoTechProject

D:\TeoTechProject>git init
Initialized empty Git repository in D:/TeoTechProject/.git/
```

2.2. Dự án các nhân VienVongProject

`cd VienVongProject → git init`

```
D:\>mkdir VienVongProject

D:\>cd VienVongProject

D:\VienVongProject>git init
Initialized empty Git repository in D:/VienVongProject/.git/
```

3. Cấu hình Git theo từng dự án (local config)

Git có 3 mức cấu hình: --system, --global, --local

Để mỗi dự án dùng thông tin khác nhau, phải dùng --local (chỉ áp dụng cho repository hiện tại).

4. Cấu hình cho dự án công ty TeoTechProject

- Di chuyển thư mục vào dự án

`cd TeoTechProject`

- Cấu hình tên và email công ty

`git config --local user.name "Nguyen Van Teo"`

git config --local user.email nvteo@teotech.com

- Kiểm tra lại cấu hình

git config --local --list

```
D:\TeoTechProject>git config --local user.name "Nguyen Van Teo"

D:\TeoTechProject>git config --local user.email nvteo@teotech.com

D:\TeoTechProject>git config --local --list
core.repositoryformatversion=0
core.filemode=false
core.bare=false
core.logallrefupdates=true
core.symlinks=false
core.ignorecase=true
user.name=Nguyen Van Teo
user.email=nvteo@teotech.com
```

5. Cấu hình cho dự án cá nhân VienVongProject

- Di chuyển thư mục vào dự án

cd VienVongProject

- Cấu hình theo tên và email cá nhân

git config --local user.name "Nguyen Van Teo"

git config --local user.email nvteo@gmail.com

- Kiểm tra lại cấu hình

git config --local --list

```
D:\VienVongProject>git config --local user.name "Nguyen Van Teo"

D:\VienVongProject>git config --local user.email nvteo@gmail.com

D:\VienVongProject>git config --local --list
core.repositoryformatversion=0
core.filemode=false
core.bare=false
core.logallrefupdates=true
core.symlinks=false
core.ignorecase=true
user.name=Nguyen Van Teo
user.email=nvteo@gmail.com
```


8.2.2. Git thực hành 2 – Các khu làm việc của Git

Bài tập 2.1: Thực hành lại các lệnh, tạo các thư mục, tập tin như trong bài học.

Kết quả thực hành:

1. Kiểm tra trạng thái ban đầu:

```
D:\TeoShop>git status
On branch master

No commits yet

nothing to commit (create/copy files and use "git add" to track)

D:\TeoShop>
```

2. Tạo tập tin index.js và kiểm tra trạng thái:

Tập tin index.js hiển thị ở mục “Untracked files” (chưa được Git theo dõi, vì mới chỉ nằm ở thư mục làm việc).

```
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        index.js

nothing added to commit but untracked files present (use "git add" to track)
```

3. Đưa tập tin vào khu tạm (Staging Area):

Kết quả git status: tập tin index.js chuyển sang mục "Changes to be committed" (đã sẵn sàng để commit)

```
D:\TeoShop>git add index.js

D:\TeoShop>git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   index.js
```

4. Quan sát 3 khu vực làm việc:

- Thư mục làm việc (Working Directory): là thư mục TeoShop (không bao gồm .git)
- Khu tạm (Staging Area): tập tin .git\index chứa thông tin về index.js đã sẵn sàng commit
- Kho chứa (Repository): thư mục .git lưu trữ lịch sử phiên bản

Bài tập 2.2: Thực hành gỡ bỏ khỏi khu tạm, viết thông điệp commit, .gitignore.

Kết quả thực hành:

1. Gỡ bỏ index.js khỏi khu tạm:

Kết quả: index.js quay lại trạng thái “Untracked files”

```
D:\TeoShop>git rm --cached index.js
rm 'index.js'
```

2. Thực hành với .gitignore

Tạo tập tin test.log trong thư mục

Tạo tập tin .gitignore với nội dung:

```
.gitignore
1  node_modules/
2  dist/
3  .env
4  *.log
5  *.tmp
```

Kết quả git status: test.log không xuất hiện trong danh sách theo dõi. Chỉ có .gitignore, index.js là tập tin mới cần được theo dõi.

```
Untracked files:
(use "git add <file>..." to include in what will be committed)
.gitignore
index.js
```

3. Viết thông điệp commit:

```
D:\TeoShop>git commit -m"Add .gitignore to exclude log files"
```

8.2.3. Git thực hành 3 – Nguyên tắc làm việc của Git

Bài tập 3a. Quy trình làm việc và các trạng thái của tập tin trong Git.

Mục tiêu: Thực hành quy trình Thư mục làm việc → Khu tạm → Kho chứa. Phân biệt trạng thái: Untracked, Staged, Committed. Sử dụng thành thạo: git init, git status, git add, git commit và git rm --cached.

Kết quả thực hành:

Bước 1. Khởi tạo dự án và kho chứa

```
C:\Data\Lab\HK 2 2025 - 2026\Software Technology\LabGit>git init  
Initialized empty Git repository in C:/Data/Lab/HK 2 2025 - 2026/Software Technology/LabGit/.git/
```

Bước 2. Thao tác tại thư mục làm việc

Tạo tập tin readme.txt

git status → tập tin hiển thị ở mục “Untracked files”

```
C:\Data\Lab\HK 2 2025 - 2026\Software Technology\LabGit>git status  
On branch master  
  
No commits yet  
  
Untracked files:  
  (use "git add <file>..." to include in what will be committed)  
      readme.txt  
  
nothing added to commit but untracked files present (use "git add" to track)
```

Bước 3. Đưa tập tin vào khu tạm

git add readme.txt

Tập tin nằm trong mục “Changes to be committed” → Trạng thái Staged

```
C:\Data\Lab\HK 2 2025 - 2026\Software Technology\LabGit>git add readme.txt  
  
C:\Data\Lab\HK 2 2025 - 2026\Software Technology\LabGit>git status  
On branch master  
  
No commits yet  
  
Changes to be committed:  
  (use "git rm --cached <file>..." to unstage)  
      new file:   readme.txt
```

Bước 4. Trải nghiệm việc gỡ bỏ khỏi khu tạm

`git rm --cached readme.txt`

`git status` → Tập tin quay lại trạng thái Untracked

```
C:\Data\Lab\HK 2 2025 - 2026\Software Technology\LabGit>git rm --cached readme.txt
rm 'readme.txt'

C:\Data\Lab\HK 2 2025 - 2026\Software Technology\LabGit>git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        readme.txt

nothing added to commit but untracked files present (use "git add" to track)
```

Bước 5. Lưu phiên bản vào kho chứa

`git commit -m "Khoi tao du an voi tap tin readme"`

`git status` → Tập tin đã ở trạng thái Committed (đã lưu vào Kho chứa)

```
C:\Data\Lab\HK 2 2025 - 2026\Software Technology\LabGit>git commit -m "Khoi tao du an voi tap tin readme"
[master (root-commit) d90b099] Khoi tao du an voi tap tin readme
1 file changed, 1 insertion(+)
create mode 100644 readme.txt

C:\Data\Lab\HK 2 2025 - 2026\Software Technology\LabGit>git status
On branch master
nothing to commit, working tree clean
```

Bài tập 3b. Rèn luyện kỹ năng phân tích

Yêu cầu: Quan sát sự khác biệt giữa các trạng thái.

Kết quả thực hành:

Tình huống 1: Chỉnh sửa tập tin đã Commit

- Mở readme.txt, thêm dòng nội dung mới và lưu

```
hello Phan Trung Hieu|
```

- `git status` → tập tin hiển thị ở mục "Changes not staged for commit"

- Phân tích: Tập tin không hiển thị là "Untracked" vì Git đã từng theo dõi nó (đã commit trước đó). Khác biệt: tập mới tạo (chưa từng commit) → Untracked; tập đã commit nhưng bị sửa → Modified/Changes not staged

```
C:\Data\Lab\HK 2 2025 - 2026\Software Technology\LabGit>git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   readme.txt

no changes added to commit (use "git add" and/or "git commit -a")
```

Tình huống 2: So sánh nội dung (git diff)

- git diff → hiển thị sự khác biệt giữa nội dung cũ và mới
 - Phân tích: git diff so sánh giữa Thư mục làm việc và Khu tạm (Staging Area).
- Nếu muốn so sánh giữa Khu tạm và Kho chứa, dùng git diff –staged

```
C:\Data\Lab\HK 2 2025 - 2026\Software Technology\LabGit>git diff
diff --git a/readme.txt b/readme.txt
index b6fc4c6..fc82f68 100644
--- a/readme.txt
+++ b/readme.txt
@@ -1,1 @@
-hello
\ No newline at end of file
+hello Phan Trung Hieu
\ No newline at end of file
```

Bài tập 3c. Rèn kỹ năng tổng hợp: “Thiết kế quy trình xử lý sự cố”

Yêu cầu: Đóng vai trưởng nhóm kỹ thuật, đưa ra giải pháp cho các tình huống.

Kết quả thực hành:

Tình huống 1: "Lỡ tay đưa nhầm tập tin quan trọng"

- Tạo tập tin matkhau.txt (giả sử chứa mật khẩu), vô tình gõ git add .
- Giải pháp: Sử dụng git rm --cached matkhau.txt để đưa tập tin ra khỏi Khu tạm mà không xóa tập tin khỏi Thư mục làm việc. Sau đó thêm matkhau.txt vào .gitignore để Git không theo dõi trong tương lai

Giải thích: Chọn git rm --cached thay vì xóa tập tin đi làm lại vì --cached chỉ gỡ bỏ khỏi Khu tạm, giữ nguyên tập tin trong Thư mục làm việc

```
C:\Data\Lab\HK 2 2025 - 2026\Software Technology\LabGit>git add .  
C:\Data\Lab\HK 2 2025 - 2026\Software Technology\LabGit>git rm --cached matkhau.txt  
rm 'matkhau.txt'
```

Tình huống 2: "Khôi phục trạng thái cũ"

- Sau khi sửa readme.txt sai, muốn quay về nội dung của lần Commit gần nhất
- Giải pháp: Sử dụng git checkout -- readme.txt (git restore readme.txt) để "đề" nội dung từ Kho chứa ngược trở lại Thư mục làm việc

Giải thích: Kho chứa (Repository) là "nguồn tin cậy" vì nó lưu trữ tất cả các phiên bản đã được commit, mỗi commit được bảo vệ bằng mã SHA-1 không thể thay đổi. Khi Thư mục làm việc xảy ra lỗi, ta luôn có thể khôi phục từ Kho chứa

```
C:\Data\Lab\HK 2 2025 - 2026\Software Technology\LabGit>git checkout -- readme.txt
```

Bài tập 3d. Rèn tư duy suy luận

Yêu cầu: Giải thích các câu hỏi dựa trên kiến thức về 3 khu vực.

Bài làm:

1. Nếu người A sửa tập tin nhưng chưa git add, người B có thấy được thay đổi đó trong Kho chứa không?

Trả lời: Không. Vì thay đổi mới chỉ nằm ở Thư mục làm việc (Working Directory) của người A. Để người B thấy được, người A phải thực hiện git add (đưa vào Khu tạm) → git commit (lưu vào Kho chứa cục bộ) → git push (đẩy lên remote repository). Khi đó, người B git pull về mới thấy được thay đổi.

2. Tại sao Git lại cần "Khu tạm" (Staging Area) mà không cho phép Commit thẳng từ Thư mục làm việc?

Trả lời: Khu tạm cho phép lập trình viên chọn lọc và tổ chức các thay đổi trước khi commit. Trong một lần làm việc, có thể có nhiều thay đổi nhưng không phải tất cả đều liên quan đến cùng một commit. Khu tạm giúp tạo ra các commit có ý nghĩa, mỗi

commit tập trung vào một vấn đề cụ thể, giúp lịch sử phiên bản rõ ràng và dễ theo dõi hơn.

8.3.Thực hành Docker

Thực hành quy trình Build – Ship – Run:

Bước 1. Chuẩn bị các thành phần

- Tạo thư mục dự án teo-docker-project
- Trong thư mục, tạo tập tin index.html có nội dung

Bước 2. Viết tập tin Dockerfile

```
# 1. Chọn hệ điều hành rút gọn có sẵn máy chủ web (Nginx)

FROM nginx:alpine

# 2. Chép mã nguồn (tập tin index.html) từ máy tính (máy của lập trình viên) vào đúng vị trí trong Image

COPY index.html /usr/share/nginx/html/index.html
```

→ Dockerfile giống như "công thức làm bánh", chứa tập hợp các câu lệnh nối tiếp nhau để Docker đọc và đóng gói thành Image

Bước 3. Thực hiện lệnh “build” để tạo Image

```
teo-docker-project>docker build -t hello-docker-image:v1 .
```

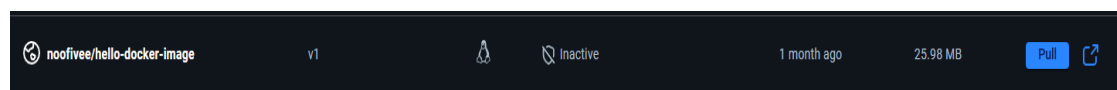
Trong đó:

docker build: lệnh tạo Image từ Dockerfile

-t hello-docker-image:v1: gán nhãn cho Image (tên: hello-docker-image, phiên bản: v1)

∴ chỉ định Docker tìm Dockerfile tại thư mục hiện hành

Bước 4. Kiểm tra kết quả trên Docker Desktop



Giải thích quá trình build: Docker đã thực hiện:

- Tải về hệ điều hành Alpine có cài sẵn web server Nginx

- Lấy tập tin index.html từ máy tính, đặt vào trong web server
- Tạo ra tập tin hello-docker-image – một tập tin tĩnh, không thay đổi, chứa tất cả những gì cần thiết để chạy trang web ở bất cứ đâu

Bước 5. Chuyển giao Image (Ship)

```
docker save -o hello-image.tar hello-docker-image:v1
```

→ Tập tin hello-image.tar được tạo ra, có thể chép qua USB hoặc gửi qua mạng

Bước 6. Khởi chạy Container (Run)

```
docker run -d -p 8080:80 --name web1 hello-docker-image:v1
```

→ Mở trình duyệt tại <http://localhost:8080> → hiển thị "Chào bác Tèo"

Kết luận về Docker: Thông qua thực hành, em đã hiểu rõ quy trình Build-Ship-Run của Docker:

- Build: Viết Dockerfile và build thành Image
- Ship: Xuất Image thành tập tin .tar hoặc đẩy lên Docker Registry
- Run: Khởi chạy Image để tạo Container, ứng dụng hoạt động trong môi trường đã được đóng gói sẵn, không phụ thuộc vào máy cài đặt

KẾT LUẬN

Công nghệ phần mềm là một lĩnh vực rộng lớn và không ngừng phát triển, đóng vai trò nền tảng cho sự thành công của các dự án phần mềm trong thực tế. Qua bài tiểu luận này, chúng ta đã có cái nhìn tổng quan và hệ thống về các khía cạnh quan trọng của môn học Công nghệ phần mềm.

Từ việc tìm hiểu khái niệm cơ bản và vòng đời phát triển phần mềm (SDLC) với 6 giai đoạn chính, chúng ta nhận thấy rằng việc phát triển phần mềm không chỉ đơn thuần là lập trình. Một kỹ sư phần mềm chuyên nghiệp cần có kiến thức và kỹ năng toàn diện, từ phân tích yêu cầu, thiết kế, lập trình, kiểm thử đến triển khai và bảo trì. Đặc biệt, các yếu tố như tính quy mô, tính dễ bảo trì và khả năng làm việc nhóm là những thách thức mà chỉ lập trình giỏi không thể giải quyết được.

Các mô hình phát triển phần mềm như Thác nước, chữ V, tiếp cận lặp và Agile/Scrum cung cấp những khung làm việc khác nhau để tổ chức quá trình phát triển. Mỗi mô hình có ưu nhược điểm riêng và phù hợp với những loại dự án cụ thể. Việc lựa chọn mô hình phù hợp dựa trên tính ổn định của yêu cầu, quy mô dự án, mức độ tham gia của khách hàng và kinh nghiệm của đội ngũ là yếu tố quan trọng góp phần vào thành công của dự án.

Quản lý dự án phần mềm là một lĩnh vực không thể tách rời khỏi công nghệ phần mềm. Với các yếu tố ràng buộc PCTS (Performance, Cost, Time, Scope), nhà quản lý dự án đóng vai trò then chốt trong việc cân bằng lợi ích của các bên liên quan, đảm bảo dự án hoàn thành đúng tiến độ, trong ngân sách và đạt chất lượng mong đợi.

Giai đoạn lấy và phân tích yêu cầu là nền tảng cho toàn bộ dự án. Việc xác định đúng và đầy đủ các yêu cầu chức năng và phi chức năng, được ghi lại trong tài liệu URS, giúp tránh được những hiểu nhầm và lãng phí trong các giai đoạn sau. Ví dụ về hệ thống bán sách trực tuyến đã minh họa rõ cách thức xác định và phân loại yêu cầu một cách có hệ thống.

Thiết kế kiến trúc và thiết kế chi tiết là hai cấp độ của quá trình chuyển đổi yêu cầu thành bản vẽ kỹ thuật. Từ kiến trúc nguyên khối đơn giản đến kiến trúc N-tầng và vi dịch vụ phức tạp, việc lựa chọn kiến trúc phù hợp ảnh hưởng sâu sắc đến khả năng

mở rộng, bảo trì và hiệu suất của hệ thống. Thiết kế chi tiết tiếp tục cụ thể hóa kiến trúc thành các chỉ dẫn lập trình cụ thể, với sự hỗ trợ của các mẫu thiết kế (Design Patterns) đã được kiểm chứng.

Cuối cùng, các công cụ hỗ trợ phát triển như Git và Docker đã trở thành những thành phần không thể thiếu trong quy trình phát triển phần mềm hiện đại. Git cung cấp khả năng quản lý phiên bản phân tán, hỗ trợ cộng tác nhóm hiệu quả và đảm bảo tính toàn vẹn của mã nguồn. Docker giải quyết bài toán "nó chạy trên máy tôi" bằng cách đóng gói ứng dụng và môi trường vào các container nhất quán, giúp quá trình triển khai trở nên đơn giản và đáng tin cậy hơn.

Tóm lại, Công nghệ phần mềm không chỉ là một môn học lý thuyết mà còn là tập hợp các nguyên lý, phương pháp và công cụ thực tiễn giúp các kỹ sư phần mềm xây dựng được những sản phẩm chất lượng cao, đáp ứng nhu cầu ngày càng phức tạp của xã hội hiện đại. Việc nắm vững những kiến thức này sẽ là hành trang quý báu cho sinh viên Công nghệ thông tin trên con đường trở thành những kỹ sư phần mềm chuyên nghiệp.

TÀI LIỆU THAM KHẢO

- [1] Lê Gia Công, "CNPM (1) - Giới thiệu," Blog Lê Gia Công, tháng 12/2025.
- [2] Lê Gia Công, "CNPM (2) - Các mô hình phát triển phần mềm," Blog Lê Gia Công, tháng 12/2025.
- [3] Lê Gia Công, "CNPM (3) - Quản lý dự án phần mềm," Blog Lê Gia Công, tháng 1/2026.
- [4] Lê Gia Công, "CNPM (4) - Lấy và phân tích yêu cầu," Blog Lê Gia Công, tháng 1/2026
- [5] Lê Gia Công, "CNPM (5) - Thiết kế kiến trúc phần mềm," Blog Lê Gia Công, tháng 1/2026
- [6] Lê Gia Công, "CNPM (6) - Thiết kế chi tiết," Blog Lê Gia Công, tháng 3/2026
- [7] Lê Gia Công, "Git thực hành (1) - Hệ thống quản lý phiên bản," Blog Lê Gia Công, tháng 2/2025
- [8] Lê Gia Công, "Git thực hành (2) - Tổng quan về Git," Blog Lê Gia Công, tháng 2/2025.
- [9] Lê Gia Công, "Git thực hành (3) - Cấu hình định danh người dùng," Blog Lê Gia Công, tháng 3/2025.
- [10] Lê Gia Công, "Git thực hành (4) - Các khu vực làm việc của Git," Blog Lê Gia Công, tháng 3/2025
- [11] Lê Gia Công, "Git thực hành (5) - Các khu vực làm việc của Git (tiếp)," Blog Lê Gia Công, tháng 3/2025
- [12] Lê Gia Công, "Git thực hành (6) - Nguyên tắc làm việc của Git," Blog Lê Gia Công, tháng 1/2026.
- [13] Lê Gia Công, "Docker (1) - Làm quen với Docker," Blog Lê Gia Công, tháng 1/2026
- [14] Lê Gia Công, "Docker (2) - Docker Image và Docker Container," Blog Lê Gia Công, tháng 3/2026